

Chương 3: Từ và Vector Từ

Oct 4, 2026 · @Le Hoang Long

Về bản dịch này

Đây là bản dịch tiếng Việt Chương 3 của sách *Natural Language Processing: Neural Networks and Large Language Models* (Tong Xiao và Jingbo Zhu, NLP Lab – Đại học Đông Bắc & NiuTrans Research, bản ngày 31/03/2026). Sách gốc phát hành theo giấy phép [CC BY-NC 4.0](#), cho phép dịch và chia sẻ phi thương mại khi ghi rõ nguồn. Bản gốc: github.com/NiuTrans/NLPBook.

Cách đọc:

- Phần chữ thường là **bản dịch** sát nghĩa nội dung gốc.
- Các khung bắt đầu bằng **Chú giải** là phần người dịch thêm vào, giải thích bằng lời đơn giản cho người đã học xong cấp 3.
- Thuật ngữ quan trọng được giữ kèm tiếng Anh trong ngoặc, vì tài liệu chuyên ngành hầu hết dùng tiếng Anh. Bảng thuật ngữ ở cuối tài liệu.
- Các trích dẫn kiểu [Tác giả, năm] được giữ nguyên; danh mục tài liệu tham khảo đầy đủ xem ở sách gốc.

Chú giải – NLP là gì? NLP (Natural Language Processing – Xử lý ngôn ngữ tự nhiên) là ngành dạy máy tính đọc, hiểu và tạo ra ngôn ngữ của con người (tiếng Việt, tiếng Anh...). Google Dịch, trợ lý ảo, ChatGPT hay Claude đều là sản phẩm của NLP. Máy tính chỉ làm việc được với **con số**, nên câu hỏi cốt lõi của chương này là: làm sao biến chữ thành số mà vẫn giữ được ý nghĩa?

Mở đầu chương

Từ là đơn vị cơ bản của ngôn ngữ [Jackendoff, 1992]. Hầu hết các hệ thống ngôn ngữ tự nhiên dùng để diễn đạt và giao tiếp đều dựa trên việc tạo ra, ghép nối và kết hợp các từ. Trước khi tìm hiểu cách các từ tạo thành những đơn vị lớn hơn, cần hiểu từ là gì. Điều này dẫn tới hai câu hỏi cơ bản:

- **Hình thức bề mặt** của một từ là gì? (tức là từ được viết ra như thế nào)
- **Ý nghĩa** của một từ là gì?

Trả lời hai câu hỏi này không dễ, vì không có quy tắc đơn giản nào mô tả đầy đủ cách từ được hình thành hay cách nghĩa của từ được xác định. Ngôn ngữ học có nhiều lý thuyết về vấn đề này, nhưng giới nghiên cứu NLP quan tâm hơn tới hai vấn đề thực tế:

- **Tách từ (Tokenization):** Cho một chuỗi ký tự, làm sao cắt nó thành dãy các từ (còn gọi là *token*) để dùng được trong các bài toán NLP phía sau?
- **Học biểu diễn từ (Word Representation Learning):** Cho một kho văn bản, làm sao biểu diễn mỗi từ ở dạng máy tính tính toán được, và mô hình NLP tính toán trên dạng biểu diễn đó ra sao?

Mục tiêu thứ nhất của chương là trình bày cách tách câu bằng phương pháp ngôn ngữ học hoặc thống kê: dùng quy tắc kinh nghiệm hoặc mô hình thống kê học từ dữ liệu để cắt chuỗi ký tự thành từ hoặc **từ con (subword)**. Mục tiêu thứ hai là cho thấy cách biểu diễn từ bằng **vector số thực**. Chương sẽ trình bày các phương pháp hiện đại để học và đánh giá các vector từ này. Trọng tâm không phải là chứng minh công thức mà là ý tưởng cốt lõi của biểu diễn vector từ – nền móng của rất nhiều hệ thống NLP. Ở các chương sau, ý tưởng này sẽ được mở rộng cho những đơn vị lớn hơn như câu.

Chú giải – Vector là gì? Ở lớp 10 bạn đã học vector trong mặt phẳng, ví dụ (3, 4). Trong máy tính, “vector” đơn giản là **một dãy số có thứ tự**, có thể dài hàng trăm số, ví dụ (0,2; -1,5; 3,0; ...). Mỗi con số gọi là một **chiều**. Ý tưởng chính của chương: mỗi từ được gán một dãy số như vậy, sao cho các từ có nghĩa gần nhau thì có dãy số “gần nhau”.

3.1 Tách từ (Tokenization)

Trong khoa học máy tính, từ *token* được dùng theo nhiều nghĩa. Ở đây ta tạm coi token là một từ theo nghĩa ngôn ngữ học, dù nó có thể là thứ khác (xem Mục 3.1.4). Trong NLP, tách từ (tokenization hay segmentation) là bài toán liên quan tới **phân tích hình thái học** [Aronoff và Fudeman, 2011]. Bộ phân tích hình thái nghiên cứu cấu trúc bên trong của từ, còn tách từ quan tâm tới cách câu được chia thành các từ.

Chú giải – Hình thái học (morphology): là ngành nghiên cứu cấu tạo bên trong của từ. Ví dụ tiếng Anh *unhappiness* = *un-* (không) + *happy* (vui) + *-ness* (danh từ hóa). Mỗi mảnh nhỏ mang nghĩa như vậy gọi là **hình vị (morpheme)**.

Ví dụ tách từ ở ba ngôn ngữ (dấu gạch chéo “/” là ranh giới từ):

Ngôn ngữ	Đầu vào	Đầu ra
Tiếng Trung	一直以来，完美的机器翻译是人类的梦想之一。	一直/以来/，/完美/的/机器翻译/是/人类/的/梦想/之一/。
Tiếng Nhật	西日本や海はく晴れて、汗ばむ暑さとなる。	西日本/や/海/は/く/晴れて/、/汗ばむ/暑さ/と/なる/。
Tiếng Anh	She said, “Deep learning is not the solution to all world's problems”.	She/said/,/“/Deep/learning/is/not/the/solution/to/all/world’s/problems”/.

Hình 3.1: Tách từ cho các ngôn ngữ khác nhau. Với tiếng Trung và tiếng Nhật – nơi giữa các từ không có dấu cách – việc này thường gọi là **phân đoạn từ (word segmentation)**.

Câu tiếng Trung có nghĩa: “Từ trước tới nay, dịch máy hoàn hảo là một trong những ước mơ của loài người.”

Có vẻ như muốn biết câu được tạo từ các từ ra sao thì ta cần biết từ được cấu tạo thế nào. Độ phức tạp còn tăng lên do sự đa dạng ngôn ngữ, khiến khó xây dựng một hệ thống chung mô tả hình thái. Ví dụ, các **ngôn ngữ đơn lập** (analytic languages, như tiếng Trung) gần như không biến hình từ, và dựa vào trật tự từ để truyền đạt nghĩa. Ngược lại, các **ngôn ngữ tổng hợp** (synthetic languages, như tiếng Pháp) có biến hình phong phú, và nghĩa của từ chịu ảnh hưởng lớn của hình thái.

Chú giải: Tiếng Việt cũng là ngôn ngữ đơn lập giống tiếng Trung: từ không đổi hình dạng (“tôi ăn”, “họ đã ăn” – chữ “ăn” giữ nguyên), nghĩa phụ thuộc vào trật tự và hư từ. Tiếng Anh thì biến hình: eat – ate – eaten – eats. Đặc biệt, tiếng Việt viết có dấu cách giữa các **âm tiết** chứ không phải giữa các **từ**: “máy tính” là một từ nhưng có dấu cách ở giữa. Vì vậy tách từ tiếng Việt cũng là một bài toán khó, tương tự tiếng Trung.

Mặt khác, chia câu thành các đơn vị nhỏ hơn rất quan trọng trong nhiều bài toán NLP, kể cả với các ngôn ngữ ít biến hình. Chẳng hạn, tiếng Trung có hình thái đơn giản và không có ranh giới từ rõ ràng. Dù có thể lấy từng chữ Hán làm đơn vị để hiểu văn bản, việc xét các đơn vị lớn hơn thường hợp lý và hiệu quả hơn. Ngay cả với những ngôn ngữ có dấu cách giữa các từ như tiếng Anh, ta vẫn cần tách từ để chuẩn hóa câu khi làm đầu vào hoặc đầu ra cho hệ thống NLP.

Trong mục này, ta bỏ qua tranh luận về việc “từ” chính xác là gì trong hình thái học và cú pháp học, mà đơn giản coi tách từ là bài toán **thêm ranh giới từ (hay token)** vào một chuỗi (xem Hình 3.1). Ta sẽ thấy câu có thể được tách theo kiểu dựa vào quy tắc kinh nghiệm hoặc theo kiểu thống kê. Lưu ý rằng mục đích là tạo ra các đơn vị giúp hệ thống NLP xử lý ngôn ngữ dễ hơn, không nhất thiết phải đúng chặt chẽ về mặt ngôn ngữ học.

3.1.1 Tách từ bằng quy tắc và kinh nghiệm

Cách phổ biến và đơn giản là xác định từng từ trong câu bằng một tập **quy tắc định sẵn**. Các quy tắc này thường dựa trên ngôn ngữ học và phản ánh hiểu biết sẵn có của ta về hình dạng của một từ. Với ví dụ tiếng Anh ở Hình 3.1, ta có thể đặt các quy tắc:

- Từ không chứa dấu cách. Vì vậy ta cắt câu thành các “ứng viên từ” theo dấu cách.
- Mọi ứng viên chỉ gồm chữ cái tiếng Anh (a-z, A-Z) là một từ.
- Mọi dấu câu (ngoặc kép, dấu phẩy, dấu chấm...) được tách riêng thành một từ.
- 's được coi là một token riêng, biểu thị sở hữu cách.

Đây có lẽ là bộ quy tắc tối thiểu cho tách từ tiếng Anh. Có thể thêm quy tắc cho nhiều hiện tượng khác, như từ có gạch nối, từ chứa chữ cái không phải tiếng Anh... Tuy nhiên không có chuẩn chung nào cho bộ quy tắc này. Trong thực tế, ta muốn một bộ quy tắc tối thiểu giải quyết được phần lớn vấn đề, và việc tách từ thường được cài đặt bằng một số **biểu thức chính quy (regular expression)**. Chi tiết xem thêm [Lawson, 2003; Friedl, 2006; Jurafsky và Martin, 2008]. Các đoạn mã tách từ có sẵn trong nhiều dự án mã nguồn mở như Moses [Koehn và cộng sự, 2007] và SacreBLEU.

Chú giải – Biểu thức chính quy: là một “ngôn ngữ mẫu” để tìm kiếm chuỗi ký tự. Ví dụ mẫu `[a-zA-Z]+` nghĩa là “một hoặc nhiều chữ cái liên tiếp”. Lập trình viên dùng nó để tìm số điện thoại, email, hay cắt câu thành từ.

Người ta cũng thường **chuẩn hóa (normalization)** văn bản trước khi tách từ để đầu vào ở dạng thống nhất. Với tiếng Anh và các ngôn ngữ dùng chữ cái, chuẩn hóa gồm: chuyển thành chữ thường, thống nhất cách mã hóa ký tự (ví dụ Unicode)... Ngoài ra, có thể đưa các dạng khác nhau của một từ về cùng một dạng. Cách đơn giản là đưa mọi dạng biến hình về **dạng gốc**. Trong ngôn ngữ học, dạng gốc này gọi là **lemma (từ nguyên mẫu)**, và quá trình đưa từ về lemma gọi là **lemmatization (chuẩn hóa về nguyên mẫu)**. Ví dụ:

- learn → learn
- learning → learn
- learns → learn

- best → good

Có những từ ứng với hai hay nhiều lemma khác nhau (thường khác từ loại). Khi đó phải chọn lemma đúng dựa vào ngữ cảnh. Nói cách khác, lemmatization **phụ thuộc ngữ cảnh**.

Chú giải: Ví dụ saw có thể là quá khứ của see (nhìn) hoặc danh từ saw (cái cưa). Muốn biết lemma là see hay saw, phải nhìn câu xung quanh.

Gần với lemmatization là **stemming (cắt gốc từ)**, biểu diễn từ bằng **gốc từ (stem)** của nó. Giống lemma, stem là một dạng cơ sở của từ. Nhưng khác lemma, stem **không nhất thiết là một từ có thật**, dù với nhiều từ lemma và stem trùng nhau. Một điểm khác nữa là stemming xử lý từng từ riêng lẻ, không cần ngữ cảnh, nên stemming **không phụ thuộc ngữ cảnh**. Có nhiều thuật toán stemming hiệu quả; một thuật toán phổ biến là **cắt hậu tố** [Porter, 1980], chỉ đơn giản bỏ các đuôi như *ing*, *ed*, *ion*...:

- remove → remov
- removing → remov
- removal → remov
- best → best

Hình 3.2 cho thấy kết quả chuẩn hóa, lemmatization và stemming cho một câu tiếng Anh:

Bước	Kết quả
Câu gốc	She said, "Deep learning is not the solution to all world's problems".
Chuẩn hóa	she said, "deep learning is not the solution to all world's problems".
Tách từ	she/said/,/"/deep/learning/is/not/the/solution/to/all/world/'s/problems"/.
Lemmatization	she/say/,/"/deep/learning/be/not/the/solution/to/all/world/'s/problem"/.
Stemming	she/said/,/"/deep/learn/is/not/the/solut/to/all/world/'s/problem"/.

Hình 3.2: Ở bước chuẩn hóa, cả câu được chuyển thành chữ thường. Ở bước lemmatization, mỗi từ được viết lại thành lemma của nó. Ở bước stemming, hậu tố của một số từ bị cắt bỏ (chú ý "solut" không phải từ có thật).

Các phương pháp trên thường được cài đặt bằng biểu thức chính quy, tra từ điển và một số mẹo kinh nghiệm. Đoạn tổng quan ngắn này có thể khiến tách từ trông đơn giản, nhưng muốn dùng được trong thực tế cần nhiều công sức hơn. Đặc biệt, với các ngôn ngữ có hệ chữ viết không dùng chữ cái, hoặc không có dấu cách giữa các từ, tách từ trở thành bài toán phức tạp và quy tắc đơn giản không còn là chiến lược tốt. Các mục tiếp theo sẽ coi tách từ là một **bài toán học máy**, trong đó cách tách câu được **học từ dữ liệu**. Các phương pháp này không phụ thuộc ngôn ngữ và áp dụng được cho nhiều bài toán kiểu phân đoạn.

3.1.2 Tách từ như một bài toán mô hình ngôn ngữ

Giờ ta chuyển sang mô hình hóa thống kê bài toán tách từ. Để dễ trình bày, mục này tập trung vào các hệ chữ viết không có ranh giới từ rõ ràng, nhưng phương pháp cũng mở rộng được cho ngôn ngữ có dấu cách. Gọi $x = x_1 \dots x_l$ là một chuỗi ký tự, và $y = y_1 \dots y_m$ là một dãy từ (token). Ta nói y là một kết quả tách từ của x nếu y định ra một cách cắt trên x . Xét câu tiếng Trung:

x = 机器翻译是人类的梦想之一。

Ta có thể định một cách cắt, ví dụ:

y = 机器翻译/是/人类/的/梦想/之一/。 = ["机器翻译" "是" "人类" "的" "梦想" "之一" "。"] (3.1)

(Nghĩa: "Dịch máy / là / loài người / của / ước mơ / một trong / .")

Như vậy, tách từ là bài toán ánh xạ x sang y . Cho một chuỗi đầu vào, đầu ra là cách cắt **có xác suất cao nhất**:

$$\hat{y} = \arg\max_y \Pr(y \mid x) = \arg\max_y \log \Pr(y \mid x) \quad (3.2)$$

Chú giải ký hiệu:

- $\Pr(y \mid x)$ đọc là "xác suất của y khi biết x " (xác suất có điều kiện – bạn đã gặp ở lớp 11-12). Ở đây: "xác suất cách cắt y là đúng, khi biết câu x ".
- **argmax** nghĩa là "chọn ra giá trị y làm cho biểu thức lớn nhất". Khác với *max* (cho biết giá trị lớn nhất là bao nhiêu), *argmax* cho biết **ai** đạt giá trị lớn nhất.
- Dấu mũ trên $\hat{y}$ nghĩa là "kết quả được chọn/ước lượng".
- Lấy **log** không làm thay đổi người thắng (vì log là hàm đồng biến), nhưng biến phép nhân nhiều xác suất nhỏ thành phép cộng, giúp máy tính tính toán ổn định hơn.

Phương trình (3.2) là mô hình dự đoán đã được nhắc nhiều lần trong sách. Tuy nhiên bài toán ở đây dễ hơn, vì mọi cách cắt hợp lệ y đều sinh ra đúng chuỗi x (ghép các từ lại là ra câu gốc). Do đó, tìm cách cắt có xác suất cao nhất quy về việc làm cực đại **xác suất tiên nghiệm** $\Pr(y)$:

$$\hat{y} = \arg\max_y \log \Pr(y) = \arg\max_y \log \Pr(y_1, \dots, y_m) \quad (3.3)$$

Dễ thấy (3.3) thực chất mô tả một bài toán **mô hình ngôn ngữ (language modeling)**. Có vài cách ước lượng xác suất đồng thời $\Pr(y_1, \dots, y_m)$. Cách đơn giản là viết $\log \Pr(y_1, \dots, y_m)$ thành tổng các log xác suất có điều kiện:

$$\log \Pr(y_1, \dots, y_m) = \log \Pr(y_1) + \log \Pr(y_2 \mid y_1) + \dots + \log \Pr(y_m \mid y_1, \dots, y_{m-1}) \quad (3.4)$$

Mỗi xác suất có điều kiện có thể được xấp xỉ bởi:

$$\Pr(y_i \mid y_1, \dots, y_{i-1}) = \Pr(y_i \mid y_{i-n+1}, \dots, y_{i-1}) \quad (3.5)$$

nghĩa là việc sinh ra y_i chỉ phụ thuộc vào $n-1$ từ đứng ngay trước. Để tính các xác suất này, ta có thể dùng phương pháp **tần suất tương đối** hoặc mạng nơ-ron (xem Chương 2).

Chú giải – Mô hình ngôn ngữ: là mô hình cho biết một câu “nghe tự nhiên” đến mức nào, dưới dạng xác suất. Ví dụ “tôi đi học” có xác suất cao hơn “học đi tôi”. Công thức (3.4) là **quy tắc nhân xác suất**: xác suất cả câu = xác suất từ đầu $\times$ xác suất từ thứ hai khi biết từ đầu $\times \dots$. Công thức (3.5) là một cách “lười” hợp lý: chỉ nhìn $n-1$ từ gần nhất thay vì cả câu (gọi là mô hình **n-gram**). Với $n = 2$ (bigram), mỗi từ chỉ phụ thuộc từ ngay trước nó. **Tần suất tương đối** nghĩa là ước lượng xác suất bằng cách đếm: ví dụ trong kho văn bản, sau chữ “đi” có 1.000 lần, trong đó 300 lần là “học”, thì $\Pr(\text{học} \mid \text{đi}) \approx 300/1000 = 0,3$.

Giờ ta có thể coi tách từ là một bài toán **học có giám sát (supervised learning)**. Quy trình như sau:

1. Chuẩn bị một số câu đã được tách từ đúng.
2. Học một mô hình ngôn ngữ $\Pr(y)$ trên các câu đã gán nhãn này.
3. Với câu mới, tìm cách tách “tốt nhất” $\hat{y}$ làm cực đại $\Pr(\hat{y})$, như trong (3.3).

Chú giải – Học có giám sát: máy học từ các ví dụ đã có sẵn “đáp án” do con người làm, giống học sinh luyện đề có lời giải, rồi tự làm đề mới.

Dù quy trình này theo đúng khuôn mẫu học có giám sát, vẫn có vài vấn đề thực tế cần giải quyết.

Thứ nhất, mô hình ngôn ngữ cần một **từ vựng (vocabulary)** định sẵn để tính xác suất cho từng token, nhưng từ mới luôn xuất hiện. Một cách xử lý là cắt chuỗi con chưa biết thành từng ký tự, tức coi ký tự là từ nếu chuỗi con đó không có trong từ vựng. Cách khác là thay mọi chuỗi con không có trong từ vựng bằng nhãn **<unk>** (unknown – không biết). Mẹo **<unk>** được dùng rộng rãi trong các mô hình ngôn ngữ hiện đại và thường có ích.

Thứ hai, mô hình ngôn ngữ ở trên **thiên vị dãy ngắn**, vì $\Pr(y_1, \dots, y_m)$ sẽ lớn nếu m nhỏ. Một cách chung để giảm thiên lệch này là thêm **phần thưởng độ dài (length reward / length bonus)**, ví dụ:

$$\hat{y} = \arg\max_y \log \Pr(y) + \lambda \cdot m \quad (3.6)$$

hoặc

$$\hat{y} = \arg\max_y \log \Pr(y)^{\frac{1}{m^\lambda}} \quad (3.7)$$

trong đó $\lambda \cdot m$ và m^λ thưởng cho các dãy dài, còn $\lambda > 0$ là một **siêu tham số (hyperparameter)** điều khiển mức độ ta dựa vào phần thưởng này khi đánh giá độ tốt của y . Điều thú vị là trong thực tế, thiên lệch độ dài không phải vấn đề lớn với tách từ, vì các kết quả tách “tốt” có độ dài chênh lệch ít. Chẳng hạn, dùng mô hình ngôn ngữ **unigram** ($n = 1$) không cần phần thưởng độ dài vẫn hoạt động tốt trong nhiều ứng dụng thực tế. Ta sẽ thấy vài ví dụ ở Mục 3.1.4.

Chú giải – Vì sao thiên vị dãy ngắn? Mỗi xác suất nằm giữa 0 và 1, nên nhân càng nhiều số càng ra kết quả nhỏ. Ví dụ $0,5 \times 0,5 = 0,25$, nhưng $0,5 \times 0,5 \times 0,5 = 0,125$. Cách cắt ít từ hơn nhân ít số hơn nên được lợi. Phần thưởng độ dài bù lại điểm cho cách cắt nhiều từ. **Siêu tham số** là con số do người làm mô hình tự chọn (không phải máy tự học), giống như “nút vặn” điều chỉnh.

Thứ ba, tìm $\arg\max$ rất khó vì số cách tách có thể tăng theo **hàm mũ**. Tuy nhiên, dùng mô hình ngôn ngữ cho phép áp dụng các thuật toán tìm kiếm hiệu quả. Ví dụ, áp dụng mô hình unigram để tách từ: với chuỗi đầu vào x , tại mỗi vị trí j ta lưu một **trạng thái** gồm xác suất

của cách tách tốt nhất trên đoạn $x_1 \dots x_j$ (ký hiệu $p(j)$) và từ cuối cùng của cách tách đó. Tại vị trí $j+1$, ta tạo trạng thái mới và tính xác suất của cách tách tốt nhất trên $x_1 \dots x_{j+1}$, bằng:

$$p(j+1) = \max_{1 \leq i \leq j} \{ p(i) \cdot \Pr(x_{i+1} \dots x_{j+1}) \} = \max_{1 \leq i \leq j} \{ p(i) \cdot \Pr(w_{[i+1, j]}) \} \quad (3.8)$$

trong đó $\Pr(w_{[i+1, j]})$ là xác suất của từ trái từ x_{i+1} tới x_j . Về mặt thuật toán, (3.8) mô tả phương pháp **quy hoạch động (dynamic programming)** với độ phức tạp thời gian $O(l^2)$ cho đầu vào độ dài l . Để có kết quả cuối, ta lần ngược từ trạng thái cuối cùng và lấy ra dãy từ trên đường đi của cách tách tối ưu.

Chú giải – Quy hoạch động: là kỹ thuật giải bài toán lớn bằng cách giải và **ghi nhớ** lời giải các bài toán con nhỏ hơn, tránh tính lại. Ở đây: muốn biết cách cắt tốt nhất cho 10 chữ đầu, ta thử mọi khả năng “từ cuối cùng bắt đầu ở đâu”, và dùng lại kết quả tốt nhất đã tính sẵn cho phần phía trước. $O(l^2)$ nghĩa là thời gian chạy tăng xấp xỉ theo bình phương độ dài câu: câu dài gấp đôi thì chậm khoảng gấp bốn – nhanh hơn rất nhiều so với thử hết mọi cách cắt ($2^{(l-1)}$ cách, tăng theo hàm mũ).

Lưu ý các phương pháp ở đây có tính tổng quát và áp dụng được cho ngôn ngữ khác. Ví dụ, khi áp dụng cho tiếng Anh, ta chỉ cần sửa nhẹ dạng đầu vào: đầu vào không phải dãy ký tự mà là dãy các mảnh nhỏ nhất được tách theo dấu câu và dấu cách. Với câu *Is this Tom's laptop?*, ta có:

$$x = [Is \ this \ Tom \ 's \ laptop \ ?] \quad (3.9)$$

Sau đó quá trình tách từ tiến hành như trong (3.2)–(3.7).

3.1.3 Tách từ như một bài toán gán nhãn chuỗi

Một cách phổ biến để ghép ký tự thành từ là **gán cho mỗi ký tự một nhãn** từ một tập nhãn định sẵn – gọi là **gán nhãn chuỗi (sequence labeling)**. Dù ở đây ta tập trung vào phân đoạn từ, khung này áp dụng được cho nhiều bài toán NLP khác như gán nhãn từ loại và nhận dạng thực thể có tên. Ý tưởng gán nhãn chuỗi đã được bàn ở Chương 1, nên ở đây chỉ trình bày cách áp dụng nó cho tách từ.

Tập nhãn dùng trong tách từ thường đơn giản. Ví dụ tập **“IB”**: nhãn **I** (Inside) chỉ một mảnh nằm bên trong từ, nhãn **B** (Begin) chỉ mảnh bắt đầu một từ. Có thể làm giàu tập nhãn bằng cách thêm nhãn **E** (End – kết thúc từ) và/hoặc tách nhãn **B** thành các nhãn con (ví dụ **B1**, **B2** chỉ mảnh thứ nhất và thứ hai của một từ) [Zhao và cộng sự, 2006]. Cho chuỗi đầu vào x và kết quả tách y , việc chuyển y thành dãy nhãn khá đơn giản. Với ví dụ ở mục trước (cách tách 机器翻译/是/人类/的/梦想/之一/。):

x	机	器	翻	译	是	人	类	的	梦	想	之	一	。
{I,B}	B	I	I	I	B	B	I	B	B	I	B	I	B
{I,B,E}	B	I	I	E	B	B	E	B	B	E	B	E	B
{I,B1,B2,E}	B1	B2	I	E	B1	B1	B2	B	B1	B2	B1	B2	B1

Chú giải: Thay vì hỏi “cắt ở đâu?”, ta hỏi với từng chữ: “chữ này là đầu từ (B) hay nằm trong từ (I)?”. Biết nhãn của mọi chữ là biết cách cắt: cứ gặp B thì bắt đầu một từ mới. Áp dụng cho tiếng Việt với từng âm tiết: “sinh/B viên/I đi/B học/B” → “sinh viên / đi / học”. (Ghi chú: dòng {I,B1,B2,E} được giữ nguyên như bản gốc.)

Vì dãy nhãn là một dạng khác của kết quả tách từ, ta có thể phát biểu lại bài toán thành tìm dãy nhãn tốt nhất cho đầu vào:

$$\hat{c} = \arg\max_c \log \Pr(c \mid x) \quad (3.10)$$

trong đó $c = c_1 \dots c_l$ là dãy nhãn độ dài l , và $\hat{c}$ là dãy nhãn có xác suất cao nhất. Nhiều phương pháp đã được đề xuất để mô hình hóa $\Pr(c|x)$. Một cách kinh điển là viết lại $\Pr(c|x)$ bằng **định lý Bayes**:

$$\hat{c} = \arg\max_c \log \frac{\Pr(x \mid c) \Pr(c)}{\Pr(x)} = \arg\max_c \log \Pr(x \mid c) + \log \Pr(c) \quad (3.11)$$

Vì $\Pr(x)$ là hằng số với mọi dãy ứng viên c , có thể bỏ qua nó khi tìm kiếm. $\Pr(x|c)$ là xác suất sinh ra đầu vào x (các *quan sát*) khi biết dãy nhãn c (các *biến ẩn*), còn $\Pr(c)$ là mô hình ngôn ngữ trên dãy nhãn. Để mô hình tính toán được, thường cần đơn giản hóa. Ví dụ, ta giả thiết **Markov**: lựa chọn c_i chỉ phụ thuộc vào c_{i-1} . Điều này dẫn tới **mô hình Markov ẩn (Hidden Markov Model – HMM)**, được dùng rộng rãi trong mô hình sinh (generative modeling) cho các bài toán NLP.

Chú giải – Định lý Bayes: công thức lật ngược xác suất có điều kiện: $\Pr(A|B) = \Pr(B|A) \cdot \Pr(A) / \Pr(B)$. Bạn có thể đã gặp trong chương trình xác suất lớp 12. Ở đây nó cho phép thay câu hỏi khó “nhãn nào khi biết chữ?” bằng hai câu hỏi dễ hơn: “nhãn này sinh ra chữ này dễ không?” và “dãy nhãn này có hợp lý không?”. **Giả thiết Markov** là giả thiết “chỉ cần nhớ bước liền trước”, giống dự báo thời tiết ngày mai chỉ dựa trên hôm nay.

Một hướng khác là **mô hình phân biệt (discriminative modeling)**. Ý tưởng phổ biến là coi gán nhãn chuỗi như một loạt bài toán **phân loại** độc lập. Ví dụ, xây dựng một bộ phân loại cục bộ dự đoán c_i dựa trên các đặc trưng quanh vị trí i . Trong các mô hình tinh vi hơn như **trường ngẫu nhiên có điều kiện (Conditional Random Fields – CRF)**, ngữ cảnh của cả chuỗi có thể được dùng để dự đoán. Bàn chi tiết các mô hình này vượt ngoài phạm vi mục; người đọc có thể xem [Kupiec, 1992; McCallum và cộng sự, 2000; Lafferty và cộng sự, 2001]. So sánh mô hình sinh và mô hình phân biệt có ở Chương 1.

Chú giải – Sinh và phân biệt: mô hình **sinh** học cách dữ liệu được “tạo ra” (nhãn nào sinh ra chữ nào). Mô hình **phân biệt** chỉ học thẳng cách “phân loại”: nhìn chữ và xung quanh rồi đoán nhãn. Giống một người học vẽ mèo để nhận ra mèo (sinh) và một người chỉ học dấu hiệu phân biệt mèo với chó (phân biệt).

3.1.4 Học từ con (Subword)

Từ thường được coi là đơn vị cơ bản của ngôn ngữ, nhưng chúng không phải yếu tố nhỏ nhất. Từ có thể phân tách thành những đơn vị nhỏ hơn có nghĩa, như hình vị. Cấu trúc phân tầng này – hình vị tạo thành từ, từ tạo thành cụm từ và câu – cho thấy vai trò trung tâm của từ trong cú pháp. Tuy nhiên, trong NLP, coi câu đơn thuần là dãy từ thường chưa tối ưu. Vấn đề chính là **dữ liệu thưa (data sparsity)** do các từ hiếm gây ra, làm việc huấn luyện mô hình kém hiệu quả. Ví dụ, từ *uncopyrightable* (không thể đăng ký bản quyền) hiếm khi xuất hiện trong kho văn bản. Hệ thống NLP có thể coi nó là từ **ngoài từ vựng (out-of-vocabulary)**, dù nghĩa của nó suy ra được bằng cách tách thành *un*, *copy*, *right* và *able*. Vấn đề khác là các chuẩn tách từ dựa trên ngôn ngữ học hạn chế khả năng mô hình tự học đơn vị phân đoạn tối ưu theo nghĩa học máy. Vì vậy, tìm các đơn vị “từ con” – dù không hẳn đúng về ngôn ngữ học nhưng phù hợp hơn cho hệ thống học máy – thường hiệu quả hơn.

Chú giải – Vì sao cần từ con? Từ vựng của một ngôn ngữ gần như vô hạn (tên riêng, từ mới, từ ghép, lỗi chính tả...). Nếu mỗi từ là một đơn vị, mô hình sẽ gặp rất nhiều từ “lạ” chưa thấy bao giờ. Nếu cắt từ thành các mảnh nhỏ hay **gặp** (như *un-*, *-able*, *-ing*), thì từ lạ vẫn ghép được từ các mảnh quen. Đây chính là cách các mô hình lớn như ChatGPT hay Claude đọc văn bản: chúng nhìn thấy các *token* là từ con, không phải từ nguyên vẹn.

1. Byte Pair Encoding (BPE – Mã hóa cặp byte)

BPE là một trong những phương pháp thành công nhất để học đơn vị từ con từ một tập các dãy từ [Sennrich và cộng sự, 2016]. Dù BPE bắt nguồn từ **nén dữ liệu** [Gage, 1994], trong NLP nó thường được dùng để giải quyết bài toán **từ vựng mở**. Ý tưởng cơ bản: lặp đi lặp lại việc thay **cặp byte xuất hiện nhiều nhất** trong dữ liệu bằng một byte mới. Kết quả là các byte phổ biến thường được gộp thành chuỗi dài hơn, còn byte hiếm thì đứng riêng như các đơn vị độc lập. Đầu ra của BPE là một từ vựng byte dùng để mã hóa dữ liệu mới.

Trong NLP, một “byte” gần tương ứng với một ký tự. Mỗi mục trong từ vựng là một dãy ký tự, gọi là **ký hiệu (symbol)** hay **từ con (subword)**. BPE bắt đầu bằng việc cắt văn bản thành dãy ký tự, ví dụ chèn dấu cách sau mỗi chữ cái tiếng Anh hoặc dấu câu. Tuy nhiên cách này thường cho ra dãy quá dài. BPE không giới hạn độ dài đầu vào, nhưng cách thường dùng là **không cho ký hiệu vượt qua ranh giới từ** để tính toán hiệu quả. Vì vậy, ta biểu diễn văn bản thành danh sách các từ đã tách ký tự, kèm số lần xuất hiện của mỗi từ. Ví dụ danh sách từ:

```
f l o w # : 2
b l o w # : 2
f l a t # : 1
f l a g # : 4
```

trong đó # là ký hiệu đặc biệt đánh dấu cuối từ. (Thay vì coi # là ký hiệu riêng, có thể gán nó vào ký tự cuối của từ, ví dụ “w#”, “t#”, “g#” biểu thị các ký tự đứng ở cuối từ.)

Chú giải: “flow” (dòng chảy) xuất hiện 2 lần, “blow” (thổi) 2 lần, “flat” (phẳng) 1 lần, “flag” (lá cờ) 4 lần.

Từ danh sách này, ta có từ vựng ban đầu (ký tự : số lần xuất hiện):

```
f : 7    b : 2
l : 9    a : 5
o : 4    t : 1
w : 4    g : 4
# : 9
```

Rồi đếm số lần xuất hiện của từng **cặp ký hiệu liên nhau (bigram)**:

```
f l : 7    a g : 4
l a : 5    g # : 4
l o : 4    b l : 2
o w : 4    a t : 1
w # : 4    t # : 1
```

Ta gộp cặp xuất hiện nhiều nhất là “f l” thành ký hiệu mới “fl”, và thay mọi chỗ “f l” trong danh sách bằng “fl”:

```
fl o w # : 2
b l o w # : 2
fl a t # : 1
fl a g # : 4
```

Đồng thời thêm “fl” vào từ vựng (fl : 7). Quá trình này lặp lại. Lần này ta gộp “fl a” thành ký hiệu mới “fla”:

```
fl o w # : 2
b l o w # : 2
fla t # : 1
fla g # : 4
```

và từ vựng có thêm “fla : 5”. Việc gộp có thể lặp lại một số lần định trước. Gộp càng nhiều lần, từ vựng càng lớn. Các mục trong từ vựng cuối cùng được sắp xếp lại theo tần suất. Ví dụ, nếu đặt số phép gộp là 6, ta có từ vựng:

```
l : 9      fla : 5      ow# : 4
# : 9      o : 4       flag : 4
f : 7      w : 4       flag# : 4
fl : 7     g : 4       b : 2
a : 5      ow : 4      t : 1
```

ứng với danh sách từ:

```
fl ow# : 2
b l ow# : 2
fla t # : 1
flag# : 4
```

Chú giải – Tóm tắt BPE bằng lời: (1) Cắt mọi từ thành từng chữ cái. (2) Đếm xem cặp chữ cái liền nhau nào hay gặp nhất. (3) Dán cặp đó thành một “mảnh” mới. (4) Lặp lại bước 2-3 đến khi đủ số mảnh mong muốn. Giống như khi chơi xếp hình, các miếng hay đi cùng nhau dần được dán sẵn thành khối lớn.

Có từ vựng BPE như trên, ta dùng nó để tách từ mới. Việc tách từ con làm theo đúng thủ tục gộp cặp như khi xây từ vựng. Cho một từ vựng BPE, trước hết ta cắt văn bản đầu vào thành các ký tự. Rồi xét từng cặp ký hiệu liền nhau trong dãy, và gộp cặp có tần suất cao nhất trong từ vựng. Lặp lại cho tới khi không gộp được nữa. Ví dụ với văn bản:

tow a flag (kéo một lá cờ)

Trước hết nó được chuyển thành dãy ký tự:

```
t o w # a # f l a g #
```

Dùng từ vựng BPE vừa có, ta thực hiện các phép gộp:

```
t o w # a # f l a g #
--(f l ⇒ fl)-->   t o w # a # fl a g #
--(fl a ⇒ fla)-->  t o w # a # fla g #
--(o w ⇒ ow)-->   t ow # a # fla g #
--(ow # ⇒ ow#)--> t ow# a # fla g #
...
-->                t ow# a # flag#
```

Dãy từ con này có thể làm đầu vào và/hoặc đầu ra cho một bài toán NLP như dịch máy. Đôi khi ta muốn ghép từ con lại thành từ. Điều này đơn giản: giữ dấu cách sau mỗi ký hiệu #, và xóa mọi dấu cách khác cùng các ký hiệu #. Cũng lưu ý rằng phương pháp BPE mô tả ở đây cần đầu vào **đã được tách từ sơ bộ**, tức cần một bộ tiền tách từ (pre-tokenizer) để cắt thô câu đầu vào. Có thể dùng các phương pháp ở Mục 3.1.1–3.1.3 cho việc này.

Chú giải: Chú ý từ “tow” chưa từng có trong dữ liệu học, nhưng BPE vẫn biểu diễn được nó bằng hai mảnh quen “t” + “ow#”. Đây chính là lợi ích của từ con: không có từ nào là “hoàn toàn lạ”.

2. WordPiece

WordPiece rất giống BPE: đầu tiên chia văn bản thành các ký hiệu nhỏ nhất, rồi dần gộp các ký hiệu liên nhau thành ký hiệu lớn hơn [Schuster và Nakajima, 2012]. Khác biệt duy nhất là **cách chọn cặp để gộp**. Ở mỗi bước, BPE gộp cặp có tần suất cao nhất. Gọi (x_i, x_{i+1}) là một cặp trong dãy x . Quy tắc gộp của BPE là:

$$(\hat{x}_i, \hat{x}_{i+1}) = \arg\max_{i \in [1, |x|-1]} \text{count}(x_i, x_{i+1}) \quad (3.12)$$

trong đó $\text{count}(x_i, x_{i+1})$ trả về số lần cặp (x_i, x_{i+1}) xuất hiện trong kho văn bản, và $(\hat{x}_i, \hat{x}_{i+1})$ là cặp có tần suất cao nhất.

WordPiece thì dùng tiêu chí **hợp lý cực đại (maximum likelihood)** để chọn cặp. Cụ thể, nó gộp cặp làm **tăng nhiều nhất khả năng (likelihood) của dữ liệu huấn luyện**. Công thức:

$$(\hat{x}_i, \hat{x}_{i+1}) = \arg\max_i \log \frac{\Pr(x_i, x_{i+1})}{\Pr(x_i)\Pr(x_{i+1})} = \arg\max_i \left[\log \Pr(x_i, x_{i+1}) - \log \big(\Pr(x_i)\Pr(x_{i+1})\big) \right] \quad (3.13)$$

Đại lượng $\log \Pr(x_i, x_{i+1}) - \log(\Pr(x_i)\Pr(x_{i+1}))$ biểu thị mức tăng log-likelihood của kho văn bản khi thay hai ký hiệu liên nhau (x_i, x_{i+1}) bằng ký hiệu gộp $x_i x_{i+1}$. Trong thống kê, tỉ số $\Pr(a,b) / (\Pr(a)\Pr(b))$ gọi là **thông tin tương hỗ theo điểm (pointwise mutual information – PMI)** của hai biến a và b (xem Mục 3.3.1). Khái niệm này cũng gọi là *information gain* và có thể diễn giải bằng độ phân kỳ Kullback–Leibler hoặc các độ đo khác trong lý thuyết thông tin. Như vậy, áp dụng quy tắc gộp này tạo ra một chuỗi phép gộp làm tăng dần khả năng của toàn bộ dữ liệu. Kết quả là một **bảng mã (codebook)** – tức một từ vựng – để xác định dãy mã khả dĩ nhất cho văn bản.

Chú giải – BPE và WordPiece khác nhau thế nào? BPE hỏi: “cặp nào xuất hiện nhiều nhất?”. WordPiece hỏi: “cặp nào đi cùng nhau **nhiều hơn mức ngẫu nhiên nhất?**”. Ví dụ “e” và “r” rất hay gặp cạnh nhau, nhưng đó có thể chỉ vì cả hai chữ đều rất phổ biến. Còn “q” và “u” tuy ít gặp hơn nhưng hầu như luôn đi cùng nhau trong tiếng Anh – WordPiece sẽ ưu tiên gộp “qu”. Tỉ số $\Pr(a,b) / (\Pr(a)\Pr(b))$ đo đúng điều này: nếu a và b độc lập thì $\Pr(a,b) = \Pr(a)\Pr(b)$, tỉ số bằng 1 (log bằng 0); tỉ số càng lớn hơn 1 thì a và b càng “dính” nhau. WordPiece được dùng trong mô hình BERT nổi tiếng.

3. SentencePiece

Cả BPE và WordPiece đều cần văn bản đầu vào đã được tiên tách thành từ. Sự phụ thuộc này làm quy trình tách từ phức tạp hơn. SentencePiece là một khung tổng quát hơn, xử lý trực tiếp văn bản thô bằng cách coi **mọi ký tự, kể cả dấu cách**, là đơn vị cơ bản [Kudo và Richardson, 2018]. SentencePiece có thể kết hợp với nhiều thuật toán từ con, nhưng thường gắn với **mô hình ngôn ngữ unigram** (gọi tắt là phương pháp unigram). Phương pháp này xây từ vựng theo chiến lược ngược hần: thay vì xây từ dưới lên, nó **bắt đầu với một từ vựng lớn rồi dần cắt tỉa** để làm cực đại khả năng của toàn bộ dữ liệu [Kudo, 2018]. (Lưu ý: *kích thước từ vựng* thường chỉ số mục trong từ vựng, nhưng trong một số ngữ cảnh tính toán có thể chỉ tổng số byte cần để lưu từ vựng.)

Phương pháp unigram coi phân đoạn từ con là bài toán mô hình ngôn ngữ unigram, có dạng giống (3.3)–(3.4). Gọi x là dãy ký tự và y là dãy từ con ghép lại thành x . Xác suất của y là:

$$\Pr(y) = \prod_{i=1}^{|y|} \Pr(y_i) \quad (3.14)$$

Rồi viết khả năng của x theo xác suất đồng thời của x và y :

$$\Pr(x) = \sum_{y \in Y(x)} \Pr(x, y) \quad (3.15)$$

trong đó tổng lấy trên mọi cách tách có thể $Y(x)$. Vì x được suy ra chắc chắn từ y , ta có $\Pr(x, y) = \Pr(y)$. Do đó viết lại (3.15):

$$\Pr(x) = \sum_{y \in Y(x)} \Pr(y) = \sum_{y \in Y(x)} \prod_{i=1}^{|y|} \Pr(y_i) \quad (3.16)$$

Chú giải ký hiệu: Π (chữ Pi hoa) là “nhân tất cả lại”, giống Σ (Sigma hoa) là “cộng tất cả lại”. Công thức (3.14) nói: xác suất một cách cắt = tích xác suất của từng mảnh (giả sử các mảnh độc lập với nhau – đó là ý nghĩa của “unigram”). Công thức (3.16) nói: xác suất của cả câu = tổng xác suất của **mọi** cách cắt có thể. Đây là công thức xác suất toàn phần bạn học ở lớp 12.

Từ đó, log-likelihood của một tập chuỗi X là:

$$\begin{aligned} \mathcal{L}(X) &= \log \prod_{x \in X} \sum_{y \in Y(x)} \prod_{i=1}^{|y|} \Pr(y_i) \\ \Pr(y_i) &= \sum_{x \in X} \log \left(\sum_{y \in Y(x)} \prod_{i=1}^{|y|} \Pr(y_i) \right) \end{aligned} \quad (3.17)$$

(Sách gốc ký hiệu đại lượng này là $\Pr(X)$; ở đây viết $\hat{\Pr}(X)$ cho đỡ nhầm với xác suất.) Nếu coi $-\hat{\Pr}(X)$ là **hàm mất mát (loss function)**, bài toán là tìm ước lượng tốt nhất cho mỗi xác suất unigram $\Pr(y)$ để $\hat{\Pr}(X)$ lớn nhất có thể. Thoạt nhìn bài toán tối ưu này phức tạp, nhưng có nhiều công cụ mạnh để giải. Một phương pháp phổ biến là **thuật toán Kỳ vọng - Cực đại hóa (Expectation-Maximization - EM)** [Dempster và cộng sự, 1977], thường dùng khi cần tìm mô hình thống kê làm cực đại khả năng của dữ liệu. Lời giải EM cho (3.17) tương tự các bài toán NLP khác như dịch máy thống kê, xem [Brown và cộng sự, 1993]. Trong chương này, ta dùng EM như một công cụ có sẵn để ước lượng $\Pr(y)$.

Ghi chú về EM (chú thích của tác giả). Trong EM, ta coi X là quan sát, và $\Pr(X|\theta)$ là mô hình thống kê mô tả khả năng xảy ra quan sát đó; θ là tham số mô hình cần xác định. EM dựa trên mục tiêu **ước lượng hợp lý cực đại**:

$$\hat{\theta} = \arg\max_{\theta} \Pr(X \mid \theta) \quad (3.18)$$

Ở đây, các $\{\Pr(y)\}$ chính là tham số mô hình. Bỏ qua phần chứng minh, thuật toán EM gồm hai bước:

- **Bước E (Kỳ vọng):** Với ước lượng hiện tại của $\Pr(y)$ (gọi là $\Pr_t(y)$), tính xác suất hậu nghiệm $\Pr_t(y)$ cho mỗi cách cắt y theo (3.14). Rồi tính **số đếm phân số (fractional count)** của mỗi từ con y trong từ vựng V :

$$\mathrm{fcount}(y) = \sum_{x \in X} \sum_{\mathbf{y} \in Y(x)} \Big(\Pr_t(\mathbf{y}) \sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i) \Big) \quad (3.19)$$

trong đó $\delta(y, y_i)$ bằng 1 nếu $y = y_i$, ngược lại bằng 0; tổng $\sum \delta(y, y_i)$ đếm số lần y xuất hiện trong dãy từ con y .

- **Bước M (Cực đại hóa):** Với các số đếm phân số từ bước E, ước lượng lại xác suất unigram:

$$\Pr_{t+1}(y) = \frac{\mathrm{fcount}(y)}{\sum_{y' \in V} \mathrm{fcount}(y')} \quad (3.20)$$

Hai bước này lặp lại nhiều vòng cho tới khi các tham số hội tụ.

Chú giải – EM bằng lời đơn giản: Ta muốn đếm xem mỗi mảnh xuất hiện bao nhiêu lần, nhưng không biết câu được cắt thể nào; mà muốn biết cắt thể nào lại cần biết mỗi mảnh hay gặp ra sao – vòng luẩn quẩn “con gà – quả trứng”. EM giải bằng cách **đoán rồi sửa dần**: (E) với xác suất hiện tại, xem mỗi cách cắt có khả năng bao nhiêu, và đếm mảnh theo kiểu “có trọng số” (một cách cắt có xác suất 0,7 thì các mảnh của nó được tính 0,7 lần – đó là “số đếm phân số”); (M) dùng các số đếm đó tính lại xác suất. Lặp lại, kết quả tốt dần lên cho tới khi ổn định.

SentencePiece về bản chất là phương pháp “**cắt tỉa**”, loại bỏ các mục có xác suất thấp khỏi từ vựng. Nó bắt đầu với một từ vựng lớn V , ví dụ liệt kê mọi chuỗi con có độ dài giới hạn (thường loại các chuỗi vượt ranh giới từ để giảm kích thước). Sau đó chạy các bước:

1. Ước lượng xác suất cho mỗi mục y của V bằng cách tối ưu (3.17).
2. Tính mức mất mát cho mỗi mục y theo chiến lược **bỏ-từng-cái (remove-one)**: mức mất mát là lượng likelihood (3.17) bị giảm khi bỏ y khỏi từ vựng.
3. Bỏ một tỉ lệ nhất định các mục trong V (*Ghi chú người dịch*: theo tài liệu gốc của Kudo, 2018, đó là các mục mà khi bỏ đi thì likelihood giảm **ít nhất**, tức các mục ít quan trọng nhất). Ví dụ giữ lại 80% và bỏ phần còn lại.

Kết quả là một từ vựng mới cùng xác suất gán cho mỗi từ con. Có thể lặp lại quá trình này nhiều lần cho tới khi từ vựng giảm tới kích thước mong muốn.

Chú giải – Ghi chú người dịch: Câu gốc trong sách viết “remove ... entries of V with large losses” (bỏ các mục có mất mát lớn), nhưng theo đúng logic thuật toán (và bài báo của Kudo, 2018), các mục bị bỏ là các mục có mất mát **nhỏ** – bỏ đi mà mô hình gần như không bị ảnh hưởng. Người dịch đã điều chỉnh để bạn đọc không bị hiểu nhầm.

SentencePiece khác BPE và WordPiece ở chỗ nó xét **mọi cách cắt từ con** có thể cho một chuỗi (xem tổng Σ trong (3.15)). Từ góc độ học máy, đây có thể coi là một cách **chính quy hóa (regularization)**: giảm rủi ro ước lượng quá cao các tham số ứng với cách cắt tốt nhất duy nhất, vốn có thể sai. Một cách khác là chỉ xét một phần các cách cắt trong $Y(x)$ cho hiệu quả. Ví dụ, lấy mẫu k cách cắt theo $\Pr(y)$ để tạo tập ứng viên $Y(x)$.

Lưu ý SentencePiece không cần đầu vào đã tách từ. BPE và WordPiece cũng có thể xử lý văn bản thô nếu được sửa đổi, nhưng SentencePiece coi đầu cách và các dấu phân cách khác **một cách tường minh là một phần của từ con**. Hình 3.3 cho vài kết quả tách *tow a flag*:

Dãy từ con	Xác suất unigram ([từ con]: xác suất)
t/ow_/a/_flag	[t]: 0,030 · [ow_]: 0,002 · [a]: 0,041 · [_flag]: 0,001
t/ow//a/f/lag	[t]: 0,030 · [ow]: 0,005 · []: 0,113 · [a]: 0,093 · [f]: 0,041 · [lag]: 0,002
t/ow/a/fla/g	[t]: 0,030 · [ow]: 0,005 · [a]: 0,084 · [fla]: 0,003 · [g]: 0,027
tow//a/f/lag	[tow]: 0,001 · []: 0,113 · [a]: 0,093 · [f]: 0,041 · [lag]: 0,002
t/ow_/a_/flag	[t]: 0,030 · [ow_]: 0,002 · [a_]: 0,093 · [flag]: 0,001

Hình 3.3: Các cách tách khác nhau cho “tow a flag”. Mỗi từ con được gán một xác suất ước lượng bằng mô hình unigram. Dấu cách được thay bằng “_” cho dễ nhìn.

Với từ vựng và xác suất unigram đã học, ta áp dụng cho văn bản mới. Đây thực chất là bài toán **tìm kiếm**: tìm dãy từ con có xác suất cao nhất theo mô hình unigram. Vì mô hình ngôn ngữ là chủ đề được nghiên cứu kỹ trong NLP, nhiều thuật toán tìm kiếm áp dụng trực tiếp được – ví dụ phương pháp ở Mục 3.1.2.

Chú giải – Thử tính một chút: Xác suất của cách cắt cuối cùng “t/ow_/a_/flag” = $0,030 \times 0,002 \times 0,093 \times 0,001 \approx 5,6 \times 10^{-9}$. Mô hình sẽ tính tích này cho mọi cách cắt rồi chọn cách cho tích lớn nhất (dùng quy hoạch động như ở Mục 3.1.2 để khỏi phải thử từng cách một).

3.2 Biểu diễn vector cho từ

Từ tự thân mang nghĩa. (Dù chương đã bàn tới nhiều đơn vị như từ con, phần còn lại vẫn dùng từ làm đơn vị cơ bản; các phương pháp sau đây áp dụng được cho ký tự, từ con...) Theo nghĩa rộng nhất, nghĩa của một từ đến từ cách nó được hiểu. Đó là thứ nằm sau hình thức bề mặt của từ, nhưng người nói ngôn ngữ đó đều hiểu được.

Sách gốc minh họa bằng vài câu trong một bài thơ thiếu nhi tiếng Anh [Knight, 2018] kể về một chú chim sẽ nhỏ đậu trên chiếc xe cút kít, ríu rít gọi bạn bè, trong khi một con mèo nhe nanh vuốt lao tới rình nó. (Người dịch chỉ tóm tắt nội dung, không chép lại nguyên văn bài thơ vì đây là tác phẩm của một tác giả khác.)

Các từ trong bài thơ không phải chuỗi ký tự ngẫu nhiên; chúng gợi lên những khái niệm cụ thể. Ví dụ *little* nghĩa là nhỏ bé, *sparrow* là một loài chim, *friends* là những người ta quý và tin tưởng. Tuy nhiên, với NLP, nghĩa của từ (word sense) không thể chỉ là một khái niệm trừu tượng trong đầu; nó phải được biến thành **dạng máy tính đọc và tính toán được**.

3.2.1 Biểu diễn one-hot

Cách tự nhiên để biểu diễn nghĩa của từ là dùng ngôn ngữ mô tả nó. Ví dụ, tra các từ trên trong từ điển (sách gốc dùng từ điển Cambridge) để có số thứ tự và định nghĩa. Người dịch tóm lược nghĩa như sau:

Từ	Số thứ tự	Nghĩa (tóm lược)
cat	511	con mèo: thú nhỏ có lông, bốn chân, đuôi và móng vuốt, thường nuôi làm cảnh hoặc bắt chuột
her	5220	cô ấy / nó (giống cái), dùng làm tân ngữ để chỉ người hoặc con vật giống cái vừa được nhắc tới
jaws	6186	hàm, miệng kể cả răng
ground	6402	mặt đất
sparrow	8331	chim sẻ: loài chim nhỏ phổ biến, màu nâu xám
wheelbarrow	9954	xe cút kít: thùng hở có một bánh phía trước và hai tay cầm phía sau, dùng để chở đồ, nhất là trong vườn

Để biểu diễn một từ, ý đơn giản nhất là thay nó bằng **số thứ tự** trong từ điển. Khi đó mỗi từ là một con số duy nhất. Một dạng tương đương là **biểu diễn one-hot**: một vector có số chiều bằng kích thước từ vựng, trong đó chỉ phần tử ở vị trí số thứ tự của từ bằng 1, mọi phần tử khác đều bằng 0. Ví dụ, từ *sparrow* (số 8331) có vector one-hot **1_sparrow**:

```
[ 0 0 ... 0 1 0 ... 0 0 ]
      ↑
    vị trí 8331
```

Chú giải: “One-hot” nghĩa là “chỉ một vị trí được bật”. Hình dung một dãy 10.000 bóng đèn, mỗi bóng ứng với một từ; để chỉ từ “sparrow”, ta bật đúng bóng số 8331 và tắt hết các bóng khác.

3.2.2 Biểu diễn phân tán (Distributed Representation)

Tuy nhiên, biểu diễn one-hot chỉ cho biết “**danh tính**” của từ chứ không cho biết **từ đó là gì**. Một vấn đề rõ ràng là mọi từ đều **trực giao** với nhau. Vì tính trực giao này, không thể dùng toán học để đo mối quan hệ ngữ nghĩa, kể cả giữa những từ rất gần nghĩa trong ngôn ngữ tự nhiên. Điều ta mong muốn là một mô hình mô tả từ bằng các **thuộc tính đo được**, và mức độ gần nghĩa được thể hiện một cách tự nhiên.

Chú giải – Trực giao: Hai vector trực giao là hai vector vuông góc, tích vô hướng bằng 0 (bạn học ở lớp 10). Với one-hot, “mèo” = (1,0,0,...) và “chó” = (0,1,0,...): tích vô hướng = 0. “Mèo” và “máy bay” cũng cho tích vô hướng = 0. Tức là theo one-hot, mèo giống chó cũng như mèo giống máy bay – hoàn toàn không phản ánh được ý nghĩa.

Một cách làm điều đó là bổ sung **định nghĩa** của từ vào biểu diễn. Lại xét từ *sparrow*. Trong từ điển, nghĩa của nó là *a common, small, gray-brown bird* (một loài chim nhỏ, phổ biến, màu nâu xám). Dùng các phương pháp tách từ và chuẩn hóa ở Mục 3.1.1, đoạn này trở thành dãy từ:

[a common , small , gray - brown bird]

Rồi ta biến dãy này thành vector theo **mô hình túi từ (bag-of-words)** (xem Chương 1):

```
[ 0 0 ... 1 ... 1 ... 1 ... 1 ... 1 ... 1 ... 1 ... 1 ... 1 ... 0 0
 ]
      ↑      ↑      ↑      ↑      ↑      ↑      ↑      ↑
      ,      -      a      bird    brown    common    gray    small
```

trong đó một phần tử bằng 1 nếu từ tương ứng có mặt, ngược lại bằng 0. Cách này cho phép các từ **chia sẻ nội dung** với nhau. Ta nói hai từ giống nhau nếu vector của chúng có phần trùng nhau. Xét từ mới *cuckoo* (chim cu cu), định nghĩa trong từ điển đại ý: một loài chim xám có tiếng kêu hai nốt nghe giống tên của nó. Dễ thấy *sparrow* và *cuckoo* có điểm chung, vì cả hai đều đánh dấu chiều “bird” bằng 1, và độ tương đồng giữa hai vector lớn hơn 0. (Độ tương đồng của hai vector có thể đo bằng **cosin của góc** giữa chúng.)

Chú giải – Túi từ và độ tương đồng cosin: “Túi từ” nghĩa là bỏ hết các từ vào một cái túi, chỉ quan tâm có từ nào, không quan tâm thứ tự. **Cosin của góc** giữa hai vector a, b được tính bằng $\cos(\theta) = (a \cdot b) / (|a| \cdot |b|)$ – công thức bạn đã học ở lớp 10. Giá trị gần 1 nghĩa là hai vector cùng hướng (rất giống nhau), bằng 0 là vuông góc (không liên quan).

Coi từ là vector số cho ta một công cụ tổng quát để biểu diễn từ theo nhiều cách khác nhau. Ta thậm chí không cần giải thích vector từ theo góc độ ngữ nghĩa. Ví dụ, có thể thêm một chiều đánh dấu từ có thuộc một loại từ (từ loại) nào đó hay không. Theo nghĩa rộng, ta có thể định nghĩa một hàm tùy ý cho mỗi phần tử của vector và coi đầu ra của hàm là một **đặc trưng (feature)** mô tả từ. Một cải tiến đơn giản là dùng hàm **đếm số lần xuất hiện** thay vì hàm nhị phân chỉ đánh dấu có/không. Các hàm đặc trưng khác có ở Mục 3.3.

Lưu ý không nhất thiết các hàm đặc trưng phải có ý nghĩa ngôn ngữ học, dù các nhà nghiên cứu NLP thường quan tâm tới thiết kế dựa trên ngôn ngữ học. Dạng tổng quát hơn của biểu diễn từ đơn giản là một **vector số thực nhiều chiều**. Nó thường được gọi là **biểu diễn phân tán (distributed representation)** của từ, hay **word embedding (nhúng từ)**. Ví dụ, từ *sparrow* có thể biểu diễn bằng vector:

[1,9 -7 3 -1,2 ... 2,01 -2,05]

Từ góc độ học máy, vector này có thể mô tả một số **thuộc tính ẩn** của từ. Các thuộc tính này có thể con người không giải thích được, nhưng **học được từ dữ liệu**. Một thách thức khi học biểu diễn như vậy là khó đo “độ tốt” của một vector. Nói chung, hỏi biểu diễn phân tán của *một* từ có tốt hay không là vô nghĩa. Thay vào đó, ta muốn biết biểu diễn của *một nhóm* từ có hành xử hợp lý không. Ví dụ, ai cũng tin rằng từ giống nhau nên có biểu diễn giống nhau. Vì vậy, quan hệ giữa các từ thường được coi là một kiểu “**khoảng cách**” giữa các biểu diễn trong không gian vector. Điều này dẫn tới nhiều phương pháp trực quan hóa và đánh giá biểu diễn từ, được bàn kỹ ở Mục 3.6.

Chú giải – Word embedding: “Embedding” (nhúng) nghĩa là đặt mỗi từ vào một điểm trong một không gian nhiều chiều (ví dụ 300 chiều). Ta không thể vẽ không gian 300 chiều, nhưng có thể hình dung giống bản đồ: các từ về động vật nằm một khu, các từ về thức ăn nằm một khu khác. Mỗi con số trong vector giống như một “tọa độ”. Khác với vector túi từ ở trên (rất dài, toàn số 0 và 1), embedding **ngắn và dày đặc** (mọi phần tử đều là số thực khác nhau), và các con số do máy **tự học**, không do người đặt ra.

Thông thường, biểu diễn từ không hoạt động riêng lẻ trong hệ thống NLP mà là **trạng thái trung gian** của một mô hình. Cách chuẩn trong NLP để học biểu diễn phân tán của từ là coi nó như **sản phẩm phụ** của việc huấn luyện một hệ thống “lớn”. Nghĩa là mô hình biểu diễn là một thành phần của hệ thống, và được tối ưu cùng các thành phần khác khi hệ thống được huấn luyện. Điều này gợi ra một hướng học biểu diễn hứa hẹn: mô hình biểu diễn được học như một mô hình con trong một hệ thống để huấn luyện, rồi có thể **cắm vào** một hệ thống hoàn toàn khác. Ví dụ, trong mô hình ngôn ngữ nơ-ron, ta buộc mô hình ánh xạ mỗi vector one-hot đầu vào thành một biểu diễn phân tán số thực, ít chiều. Các biểu diễn

này được đưa vào một mạng nơ-ron dự đoán phân phối xác suất trên từ vựng. Hàm ánh xạ (hàm embedding) được huấn luyện để giảm thiểu mất mát của mô hình ngôn ngữ trên dữ liệu (xem Mục 3.4). Khi dùng hàm embedding đã học, ta bỏ mọi phần khác của mô hình ngôn ngữ và chỉ dùng hàm này để sinh biểu diễn cho mỗi từ trong các bài toán phía sau. Một chiến lược khác là thiết kế mô hình **riêng cho bài toán học biểu diễn từ**. Các hệ thống này thường không nhằm giải bài toán NLP tiêu chuẩn, mà tập trung vào các vấn đề cụ thể của học biểu diễn từ, như mô hình hóa trực tiếp quan hệ giữa các từ (xem Mục 3.5).

3.2.3 Tính kết hợp và tính ngữ cảnh

Dù thảo luận ở đây giới hạn trong học biểu diễn từ cho NLP, nghiên cứu nghĩa của từ vốn là một nhánh truyền thống của ngôn ngữ học. Chẳng hạn, trong **ngữ nghĩa học từ vựng (lexical semantics)**, các nhà nghiên cứu tìm hiểu nghĩa của từ được định nghĩa thế nào và chúng kết hợp thành nghĩa của câu ra sao. Trong lịch sử, biểu diễn từ bằng máy tính không bám sát ngữ nghĩa học ngôn ngữ, mà tập trung vào việc chuyển đơn vị ngôn ngữ thành dạng tính toán được. Tuy nhiên, vì ngữ nghĩa là nền tảng của hiểu ngôn ngữ, việc đưa các nguyên lý ngữ nghĩa vào thiết kế mô hình biểu diễn vẫn rất quan trọng. Xét hai tính chất sau:

Tính kết hợp (Compositionality). Đây là khái niệm phổ biến trong ngữ nghĩa học, logic học và các lĩnh vực liên quan. Nó thường đi kèm với **nguyên lý kết hợp**:

“Nghĩa của một biểu thức phức hợp được quyết định bởi cấu trúc của nó và nghĩa của các thành phần.” – Szabó [2020]

Nguyên lý kết hợp có mặt khắp nơi trong ngôn ngữ tự nhiên. Ví dụ, nghĩa của cụm *white cat* (mèo trắng) dễ dàng suy ra từ nghĩa của *white* (trắng) và *cat* (mèo). Tương tự, câu ghép lầy nghĩa tổng thể từ sự kết hợp các mệnh đề độc lập và liên từ. Lưu ý nguyên lý này không chỉ là một quy tắc đơn giản mô tả cách đơn vị lớn được tạo từ đơn vị nhỏ, dù đã có người cố gắng định nghĩa nó một cách hình thức [Montague, 1974]. Hiện vẫn có tranh luận về cách hiểu và mô hình hóa nguyên lý này trong các lý thuyết ngữ nghĩa.

Tuy vậy, nếu tập trung vào ứng dụng NLP và gác lại tranh luận lý thuyết, tính kết hợp vẫn là tính chất rất có giá trị cho thiết kế và đánh giá hệ thống. Khi một bài toán có tính kết hợp, nghĩa là bài toán phức tạp có thể được chia nhỏ một cách có hệ thống thành các phần dễ xử lý hơn. Với học biểu diễn từ, ta muốn các vector kết quả có tính kết hợp, phản ánh bản chất tổ hợp của ngôn ngữ. Mục 3.6 sẽ cho vài ví dụ. Chẳng hạn, biểu diễn học bởi mạng nơ-ron thường thể hiện các tính chất đại số có ý nghĩa dưới các phép toán tuyến tính, dù bản thân mô hình là phi tuyến. Tuy nhiên, tính kết hợp không phải quy luật tuyệt đối. Nó

thường bị phá vỡ ở các trường hợp như **cụm từ cố định (collocation)** và **thành ngữ (idiom)**, nơi ngôn ngữ tự nhiên không có tính kết hợp. Điều này giải thích vì sao các bài toán NLP khó đến vậy.

Chú giải: Tiếng Việt có rất nhiều ví dụ phá vỡ tính kết hợp. “Áo dài” không chỉ đơn giản là “áo” + “dài” (một chiếc áo khoác dài không phải áo dài). Thành ngữ “ăn cháo đá bát” chẳng liên quan gì tới cháo hay bát, mà chỉ người vô ơn. Máy tính nếu chỉ ghép nghĩa từng chữ sẽ hiểu sai hoàn toàn.

Tính ngữ cảnh (Contextuality). Tính ngữ cảnh thể hiện một dạng không kết hợp. Nó nói rằng một từ có thể có nhiều nghĩa, và nghĩa cụ thể do **ngữ cảnh xung quanh** quyết định. Ví dụ, từ *table* trong tiếng Anh có hai nghĩa chính:

- **Nghĩa 1:** mặt phẳng để đặt đồ lên – cái bàn. Ví dụ (người dịch tự đặt): *We had lunch at the kitchen table.* (Chúng tôi ăn trưa ở bàn bếp.)
- **Nghĩa 2:** cách sắp xếp các mục theo hàng, cột hoặc khối – cái bảng. Ví dụ: *The table below shows last year's sales.* (Bảng dưới đây cho thấy doanh số năm ngoái.)

Sách gốc đưa ra sáu câu ví dụ lấy từ từ điển Cambridge, trong đó ba câu dùng nghĩa “bàn” (bàn ăn, bàn để đồ) và ba câu dùng nghĩa “bảng” (bảng số liệu, bảng xếp hạng). Nói cách khác, *table* là **từ đa nghĩa (polysemous)**, tức là từ **mơ hồ**. Sự mơ hồ này thường được giải quyết bằng cách xem các từ lân cận. Ví dụ, khi *table* đứng sau *dinner* (bữa tối), dễ biết nó mang nghĩa 1.

Sự mơ hồ cũng xảy ra khi một từ bắt nguồn từ vài dạng hay **từ vị (lexeme)** khác nhau (hiện tượng **đồng âm – homonymy**). Ví dụ *bear* có thể là động từ (chịu đựng, mang) hoặc danh từ (con gấu). Việc xác định nghĩa của từ trong một tập nghĩa cho trước đã được nghiên cứu hàng chục năm trong NLP, gọi là **khử nhập nhằng nghĩa từ (word sense disambiguation – WSD)** [Kelly và Stone, 1975]. Tuy nhiên, bài toán biểu diễn từ ở đây khó hơn, vì ta thường **không có sẵn** một tập nghĩa định trước. Thay vào đó, ta muốn một **mô hình biểu diễn theo ngữ cảnh**, sinh ra biểu diễn của từ phụ thuộc vào ngữ cảnh. Vì vậy, cần nhận ra rằng nghĩa của từ **không cố định**. Điều này làm bài toán hơi khác với phân đầu mục: ta không còn một bảng tra cứu biểu diễn từ, mà là một mô hình tạo ra các biểu diễn khác nhau cho cùng một từ trong các ngữ cảnh khác nhau.

Chú giải: Tiếng Việt cũng vậy. “Đường” trong “đường phố” và “đường mía” là hai nghĩa hoàn toàn khác. “Bàn” trong “cái bàn” và “bàn bạc” cũng thế. Một bảng tra cứu cố định chỉ cho mỗi từ **một** vector, nên không phân biệt được. Các mô hình hiện đại như BERT, GPT sinh vector khác nhau cho “đường” tùy theo câu.

Phần còn lại của chương tập trung vào việc học vector biểu diễn từ dựa trên **sự phân bố của chúng trong ngôn ngữ thực tế**. Các mô hình theo ngữ cảnh để học biểu diễn từ dày đặc sẽ được bàn ở Chương 4–6.

3.3 Các mô hình dựa trên đếm (Count-based Models)

Ta đã coi việc tìm nghĩa của từ là bài toán học vector từ. Ở mục này, ta giả định rằng **nghĩa của một từ do môi trường mà từ đó được dùng quyết định**. Điều này thường được phát biểu thành **giả thuyết phân bố (distributional hypothesis)**: các từ có nghĩa giống nhau nếu chúng xuất hiện trong ngữ cảnh giống nhau [Harris, 1954; Firth, 1957]. Biểu diễn từ học theo giả thuyết này gọi là **biểu diễn phân bố (distributional representation)**. Để dễ đọc, sách vẫn dùng cụm *vector từ* và *biểu diễn từ*. Sau đây là vài phương pháp mô hình hóa sự phân bố của từ trong văn bản, rồi một số cải tiến.

Chú giải – Giả thuyết phân bố: Ý tưởng của nhà ngôn ngữ học Firth có thể tóm lại là: muốn biết một từ nghĩa gì, hãy xem nó hay đi cùng những từ nào. Nếu bạn gặp một từ lạ như “chôm chôm” và thấy nó hay đi với “ăn”, “ngọt”, “bóc vỏ”, “mùa hè” – giống hệt “nhãn” hay “vải” – bạn sẽ đoán được đó là một loại trái cây. Máy tính làm đúng như vậy: đếm xem từ hay đi với những từ nào.

Lưu ý thuật ngữ (chú thích của tác giả): *Biểu diễn phân bố* (distributional) và *biểu diễn phân tán* (distributed) là hai khái niệm khác nhau. Biểu diễn phân bố mô tả cách một đơn vị ngôn ngữ phân bố trong thực tế sử dụng; đối lập với nó là biểu diễn lấy từ cơ sở dữ liệu từ vựng (như định nghĩa trong từ điển). Còn biểu diễn phân tán là một vector các biến ứng với những thuộc tính ẩn của đơn vị ngôn ngữ; đối lập với nó là biểu diễn one-hot chỉ cho biết ký hiệu của từ.

3.3.1 Ma trận đồng xuất hiện (Co-occurrence Matrices)

Trong ngữ nghĩa học phân bố, từ được biểu diễn bằng các mô hình xét nhiều khía cạnh của ngữ cảnh. Các mô hình khác nhau ở cách mô hình hóa ngữ cảnh: xét ngữ cảnh rộng bao nhiêu, đếm mỗi lần xuất hiện thế nào, định nghĩa số chiều ra sao... Trong mục này, như phần lớn mô hình NLP, ta giả định biểu diễn từ được học từ một tập văn bản (tài liệu).

Một tài liệu có thể coi đơn giản là một tập hợp từ **không thứ tự** (giả định túi từ). Khi đó, mỗi lần xuất hiện của các từ là một dấu hiệu ngữ cảnh độc lập. Như vậy, sự phân bố của một từ trong ngữ cảnh được mô tả bằng **số lần nó cùng xuất hiện với các từ ngữ cảnh**. Ta

xây dựng một **ma trận đồng xuất hiện**, trong đó mỗi ô ghi số lần một mục hàng và một mục cột cùng xuất hiện. Ví dụ, xét các tài liệu sau (trích từ Wikipedia, người dịch dịch nghĩa):

Tài liệu	Nội dung (bản gốc tiếng Anh dùng để đếm; đây là dịch nghĩa)
Doc 1	Quả mọng (berry) là một loại quả nhỏ, nhiều thịt và thường ăn được.
Doc 2	Theo thuật ngữ thực vật học, quả mọng là quả đơn có hạt và thịt quả, phát triển từ bầu nhụy của một bông hoa duy nhất.
Doc 3	Từ “banana” (chuối) cũng được dùng làm tên gọi chung cho các loài cây cho ra loại quả này.
Doc 4	Hạt chuối to, cứng, có gai và dễ làm mẻ răng.
Doc 5	Chuối là loại quả thuôn dài, ăn được – về mặt thực vật học là một quả mọng – do nhiều loài cây thân thảo lớn có hoa thuộc chi Musa tạo ra.

Với mỗi cặp từ, ta đếm tổng số lần chúng cùng xuất hiện trong các tài liệu, được một ma trận gọi là **ma trận đồng xuất hiện từ-từ** (word-word hay term-term co-occurrence matrix). Đây là một phần của ma trận:

	flowering	fruit	herbaceous	...	often	plants	seeds
berry	1	3	1	...	1	1	1
terminology	0	1	0	...	0	0	1
common	0	1	0	...	0	1	0
teeth	0	0	0	...	0	0	1
banana	1	2	1	...	0	2	1
simple	0	1	0	...	0	0	1
and	0	2	0	...	1	0	2

(flowering = có hoa, fruit = quả, herbaceous = thân thảo, often = thường, plants = cây, seeds = hạt; terminology = thuật ngữ, common = chung/phổ biến, teeth = răng, simple = đơn giản/đơn.)

Trong ma trận, mỗi từ ở hàng gắn với một vector từ có $|V|$ phần tử ($|V|$ là kích thước từ vựng). Các số trong vector cho biết từ ở hàng hay đi cùng các từ ngữ cảnh khác nhau ra sao, tức từ đó **phân bố** thế nào trong các “ngữ cảnh”. Về mặt hình học, nếu hai từ có cách đồng xuất hiện giống nhau với cùng một nhóm từ ngữ cảnh, thì **góc giữa hai vector sẽ nhỏ**. (Góc giữa hai vector không phụ thuộc độ dài của chúng. Nếu các vector được chuẩn hóa, thì hai vector giống nhau nghĩa là phần lớn phần tử của chúng có giá trị gần nhau.) Ví dụ, nếu coi các từ là vector trong không gian vector, *berry* gần *banana* hơn là *teeth* (Hình 3.4). Trục giác hình học này là nền tảng của nhiều mô hình biểu diễn, sẽ gặp lại ở Chương 4 và 5.

Hình 3.4 (mô tả): Các vector từ trong không gian vector, xây dựng từ thống kê đồng xuất hiện trên dữ liệu tiếng Anh của WMT 2012. Mọi vector được chuẩn hóa và vẽ thành mũi tên. Để trực quan hóa, các vector được chiếu từ không gian nhiều chiều xuống không gian 3 chiều bằng phân tích thành phần chính (PCA). Đúng như kỳ vọng, mũi tên berry nằm gần mũi tên banana hơn mũi tên teeth: khoảng cách 1 (berry-banana) ngắn hơn khoảng cách 2 (berry-teeth).

Chú giải – Đọc ma trận: Nhìn hàng *berry* và hàng *banana*: cả hai đều có số khác 0 ở cột *flowering*, *fruit*, *herbaceous*, *plants*, *seeds* – tức đều hay đi với các từ về thực vật. Hàng *teeth* thì gần như toàn 0, chỉ có 1 ở cột *seeds*. Vì vậy vector *berry* “cùng hướng” với *banana* hơn là *teeth*. Đây là cách máy tự phát hiện “quả mọng” và “chuôi” cùng nhóm nghĩa mà không cần ai dạy.

Một vấn đề của phương pháp này là không tính tới **khoảng cách giữa các từ**, dù mức liên quan thường yếu đi khi từ ngữ cảnh ở xa. Giải pháp đơn giản là giới hạn từ ngữ cảnh trong một **cửa sổ ngữ cảnh (context window)** [Lund và Burgess, 1996]. Ví dụ, với mỗi từ trong tài liệu, ta chỉ đếm 2 từ phía trước và 2 từ phía sau nó (cửa sổ kích thước 5).

Lưu ý vector từ học bằng mô hình túi từ ở Mục 3.2 là một trường hợp đặc biệt của ma trận đồng xuất hiện: cả kho chỉ có một tài liệu, và hàm chỉ thị đánh dấu có/không thay vì đếm. Ngoài số đếm hay chỉ thị nhị phân, các phần tử của vector từ có thể được điền bằng nhiều **độ đo liên kết (association measures)**. Giá trị một phần tử có thể coi là mức độ tương ứng giữa hai từ; nó không nhất thiết là số đếm mà có thể là số thực tùy ý. Như vậy, bài toán trở thành **đo độ mạnh liên kết giữa các từ**. Thông thường, độ đo này được định nghĩa dựa trên các mô hình tương quan. Trong thống kê, **tương quan (correlation)** mô tả mức độ hai biến liên quan với nhau, đo bằng **hệ số tương quan**, như hệ số Pearson (Pearson’s r) hay hệ số tương quan hạng Spearman (Spearman’s ρ). (Một số hệ số giả định dữ liệu tuân theo phân phối nhất định; ví dụ hệ số Pearson giả định hai biến có phân phối chuẩn.) Trong NLP, độ đo được dùng rộng rãi là **thông tin tương hỗ theo điểm (PMI)** [Church và Hanks, 1990]. Gọi a và b là hai từ:

$$\mathrm{PMI}(a, b) = \log \frac{\Pr(a, b)}{\Pr(a)\Pr(b)} \quad (3.21)$$

trong đó $\Pr(a, b)$ là xác suất a và b cùng xuất hiện, còn $\Pr(a)$ (hay $\Pr(b)$) là xác suất a (hay b) xuất hiện, ước lượng bằng tần suất tương đối. (Một vấn đề của PMI là độ đo **không ổn định với từ hiếm**: nếu một từ rất hiếm tình cờ xuất hiện trong một tài liệu, PMI giữa nó và mọi từ khác trong tài liệu sẽ lớn bất hợp lý.) Cho một từ a và từ vựng các từ ngữ cảnh $\{b_1, \dots, b_{|V|}\}$, vector từ của a dựa trên PMI là:

$$\mathbf{e}(a) = \left[\mathrm{PMI}(a, b_1) \ ; \ ; \ ; \dots \ ; \ ; \ ; \mathrm{PMI}(a, b_{|V|}) \right] \quad (3.22)$$

Chú giải – PMI bằng ví dụ số: Giả sử trong kho văn bản, xác suất gặp “cà” là 0,01, gặp “phê” là 0,01. Nếu hai chữ độc lập, xác suất gặp “cà phê” cạnh nhau chỉ là $0,01 \times 0,01 = 0,0001$. Nhưng thực tế ta đếm được $\Pr(\text{cà, phê}) = 0,005$ – gấp 50 lần mức ngẫu nhiên. $\mathrm{PMI} = \log(50) > 0$: hai chữ “dính” nhau mạnh. $\mathrm{PMI} \approx 0$: không liên quan; $\mathrm{PMI} < 0$: hai từ “tránh” nhau.

Các hệ số tương quan thường dùng để kiểm định xem hai biến có liên quan (tuyến tính) hay không. Vì vậy, một cách khác là định nghĩa phần tử của vector từ là **kết quả của một kiểm định**: ví dụ phần tử (a, b) bằng 1 nếu hệ số tương quan giữa a và b vượt một ngưỡng, hoặc nếu kiểm định giả thuyết thống kê ủng hộ đủ mạnh tương quan đó.

Tuy nhiên, mô hình hóa từ bằng vector điểm tương quan giữa các từ phần nào hạn chế thông tin ngữ cảnh có thể dùng. Một cách tiếp cận vĩ mô hơn là phân tích cách từ phân bố **trên toàn bộ các tài liệu**, tạo thành **ma trận đồng xuất hiện từ-tài liệu** (term-document matrix). Với các tài liệu trên:

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
berry	1	1	0	0	1
terminology	0	1	0	0	0
common	0	0	1	0	0
teeth	0	0	0	1	0
banana	0	0	1	1	1
simple	0	1	0	0	0
and	1	1	0	2	0

Giá trị $\phi(a, d)$ là số lần từ a xuất hiện trong tài liệu d , đo độ mạnh liên kết giữa a và d . Đại lượng này gọi là **tần suất từ (term frequency - TF)**, ký hiệu $tf(a, d)$. Cũng có thể dùng hàm chỉ thị 0-1 (xem Mục 3.2). Bảng 3.1 liệt kê vài biến thể của hàm trọng số TF.

Kiểu	Công thức
Nhị phân	$tf(a, d) = 1$ nếu a xuất hiện trong d ; 0 nếu không
Số đếm	$tf(a, d) = \text{count}(a; d)$
Số đếm mũ	$tf(a, d) = \text{count}(a; d)^\alpha$
Số đếm thang log	$tf(a, d) = \log(1 + \text{count}(a; d))$
Số đếm chuẩn hóa (tần suất)	$tf(a, d) = \text{count}(a; d) / \sum_{a'} \text{count}(a'; d)$

Bảng 3.1: Các hàm trọng số tần suất từ. $\text{count}(a; d)$ đếm số lần từ a xuất hiện trong tài liệu d .

Chú giải: Dạng **log** được dùng vì một từ xuất hiện 100 lần không quan trọng gấp 100 lần từ xuất hiện 1 lần; log “nén” chênh lệch: $\log(1+1) \approx 0,69$ còn $\log(1+100) \approx 4,6$. Dạng **chuẩn hóa** chia cho tổng số từ trong tài liệu, để tài liệu dài không được lợi chỉ vì nhiều chữ.

Cấu trúc này phục vụ **hai mục đích**: mỗi **hàng** là biểu diễn vector của một từ, còn mỗi **cột** là biểu diễn vector của cả một tài liệu. Ma trận từ–tài liệu về khái niệm giống hệt mô hình túi từ dùng cho phân loại văn bản ở Chương 1, nơi bỏ thứ tự từ nhưng giữ tần suất tương đối. Ở đây ta vector hóa tài liệu bằng mô hình này trên một tập tài liệu.

3.3.2 TF-IDF

Mô hình hóa liên kết từ–tài liệu quan trọng với nhiều bài toán NLP. Một cải tiến đáng kể so với số đếm thô là sơ đồ trọng số **tần suất từ – nghịch đảo tần suất tài liệu (TF-IDF)**. Cho tập tài liệu D , TF-IDF gán điểm cho mỗi cặp từ–tài liệu (a, d) :

$$\text{tfidf}(a, d, D) = \text{tf}(a, d) \cdot \text{idf}(a, D) \quad (3.23)$$

trong đó:

- **$tf(a, d)$** là tần suất từ (Bảng 3.1). Khi $tf(a, d)$ lớn, từ a là dấu hiệu tốt cho tài liệu d ; khi nhỏ, liên kết không mạnh.

- **idf(a, D)** là **ngịch đảo tần suất tài liệu**. Nó dựa trên thực tế là từ phổ biến trong nhiều tài liệu thì ít mang thông tin. Ví dụ, trong một tập tài liệu về thể thao, *player* và *players* (cầu thủ) có mặt ở hầu hết tài liệu, nên không giúp phân biệt các tài liệu. Gọi $df(a, D)$ là số tài liệu trong D chứa từ a. Dạng phổ biến của idf là:

$$\mathrm{idf}(a, D) = \log \frac{|D|}{\mathrm{df}(a, D)} \quad (3.24)$$

Công thức này phạt các từ xuất hiện khắp kho văn bản. Tương tự, có phiên bản **làm trơn (smoothed)**:

$$\mathrm{idf}(a, D) = \log \frac{|D|}{\mathrm{df}(a, D) + 1} + 1 \quad (3.25)$$

Chú giải – TF-IDF bằng ví dụ: Có 1.000 bài báo. Từ “và” xuất hiện trong cả 1.000 bài: $\mathrm{idf} = \log(1000/1000) = 0 \rightarrow$ điểm TF-IDF luôn bằng 0, vì “và” chẳng cho biết bài nói về gì. Từ “bóng đá” chỉ xuất hiện trong 10 bài: $\mathrm{idf} = \log(100) \approx 4,6 \rightarrow$ rất “đặc trưng”. Nếu một bài nhắc “bóng đá” 5 lần, điểm của nó cao và ta đoán được chủ đề. Tóm lại: **TF** = từ này quan trọng trong bài này cỡ nào; **IDF** = từ này hiếm cỡ nào trong cả kho. Bản làm trơn cộng thêm 1 để tránh chia cho 0.

Dùng trọng số TF-IDF, ta xây ma trận từ–tài liệu với giá trị ô (a, d) là $\mathrm{tfidf}(a, d, D)$; mỗi hàng là biểu diễn vector của từ tương ứng. Theo truyền thống, TF-IDF và ma trận từ–tài liệu được dùng nhiều để biểu diễn tài liệu. Chẳng hạn, trong **truy xuất thông tin (information retrieval)**, cả truy vấn lẫn tài liệu đều được biểu diễn bằng vector cột TF-IDF, rồi tính độ liên quan bằng độ tương đồng vector. Mô hình không gian vector trong truy xuất thông tin nằm ngoài phạm vi chương; xem [Manning và cộng sự, 2008; Buttcher và cộng sự, 2016].

Chú giải: Các công cụ tìm kiếm thời kỳ đầu hoạt động đúng như vậy: biến câu bạn gõ thành vector TF-IDF, rồi tìm các trang web có vector “cùng hướng” nhất.

3.3.3 Các mô hình ít chiều (Low-Dimensional Models)

Ma trận đồng xuất hiện thường **rất nhiều chiều**. Giả sử từ vựng có 20.000 từ và có 10.000.000 tài liệu. Khi đó ma trận từ–tài liệu có $20.000 \times 10.000.000 = 2 \times 10^{11}$ phần tử. Với gánh nặng tính toán như vậy, biểu diễn mỗi từ bằng vector 10.000.000 chiều và mỗi tài liệu bằng vector 20.000 chiều là không thực tế. Ta muốn biểu diễn từ (hay tài liệu) chỉ cần một số khá nhỏ đặc trưng. Mục này bàn các cách chuẩn để chuyển ma trận nhiều chiều thành

biểu diễn ít chiều. Vì các kỹ thuật này đã được nghiên cứu sâu ở nhiều ngành [Barber, 2012; Wright và Ma, 2022], ta bỏ qua chứng minh và tập trung vào cách dùng chúng để có vector từ và tài liệu.

Chú giải: $2 \times 10^{11} = 200$ tỷ con số – cần hàng trăm GB bộ nhớ chỉ để lưu, và phần lớn là số 0 (ma trận **thưa**). **Giảm chiều** giống như tóm tắt: thay vì mô tả một người bằng 10 triệu chi tiết, ta chỉ cần vài trăm đặc điểm quan trọng nhất.

1. Phân tích ngữ nghĩa tiềm ẩn (Latent Semantic Analysis – LSA)

Trong NLP, LSA là phương pháp tìm **cấu trúc ngữ nghĩa ẩn** đằng sau liên kết từ–tài liệu [Deerwester và cộng sự, 1990; Landauer và cộng sự, 1998]. (LSA còn gọi là *latent semantic indexing – LSI*, tên hay dùng trong truy xuất thông tin.) Nó giả định cả từ và tài liệu đều biểu diễn được bằng vector ít chiều chất lọc từ ma trận đồng xuất hiện, vẫn giữ tính chất của mô hình không gian vector ban đầu (ví dụ góc giữa vector của các từ giống nhau vẫn nhỏ).

Cụ thể, LSA **phân tích ma trận đồng xuất hiện** thành tích của một ma trận biểu diễn từ, một ma trận biểu diễn tài liệu, và một ma trận thứ ba nối hai ma trận kia. Về toán học, đây là phép **phân tích giá trị kỳ dị (singular value decomposition – SVD)** [Stewart, 1993]. Gọi $M \in \mathbb{R}^{|V| \times |D|}$ là ma trận đồng xuất hiện trên từ vựng V và tập tài liệu D . SVD cho:

$$\mathbf{M} = \mathbf{P} \mathbf{\Sigma} \mathbf{Q}^T \quad (3.26)$$

với $P \in \mathbb{R}^{|V| \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, $Q^T \in \mathbb{R}^{r \times |D|}$. P và Q^T là ma trận **nửa trực giao**: các cột của P và của Q là các vector trực chuẩn (vuông góc từng đôi, độ dài 1). Do đó các cột của P và Q lần lượt là cơ sở trực chuẩn cho không gian cột và không gian hàng của M , với r là **hạng (rank)** của M . Điều này đảm bảo M được biểu diễn bằng số chiều tối thiểu cần thiết. Σ là **ma trận đường chéo**:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ \sigma_2 & \cdots & 0 & \vdots \\ & \ddots & \vdots & \vdots \\ & & \sigma_r & 0 \end{bmatrix} \quad (3.27)$$

Các phần tử đường chéo $\{\sigma_1, \dots, \sigma_r\}$ là số thực không âm, gọi là **giá trị kỳ dị (singular values)** của M , thường sắp giảm dần ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$). Viết P thành các vector cột (**vector kỳ dị trái**) và Q^T thành các vector hàng (**vector kỳ dị phải**):

$$\mathbf{M} \approx \sum_{i=1}^{r^*} \sigma_i \mathbf{p}_i \mathbf{q}_i^T \quad (3.34)$$

Vế phải của (3.34) gọi là **xấp xỉ hạng thấp (low-rank approximation)** của $\mathbf{M}$: bằng cách chọn r^* , ta xấp xỉ $\mathbf{M}$ bằng một ma trận có hạng tùy ý nhỏ hơn r . Có nhiều thuật toán thực hiện SVD [Cline và Dhillon, 2014]; phần lớn cài đặt hiện đại đều hiệu quả và dùng được như công cụ có sẵn.

Chú giải: Giống nén ảnh: giữ vài trăm “lớp” quan trọng nhất là đủ để ảnh trông gần như nguyên vẹn. Mỗi từ giờ chỉ còn r^* con số (ví dụ 300) thay vì 10 triệu. Điều thú vị: ép dữ liệu qua ít chiều buộc mô hình gom các từ có ngữ cảnh giống nhau lại, nên các từ đồng nghĩa (như “ô tô” và “xe hơi”) có vector gần nhau dù có thể chưa từng xuất hiện chung một tài liệu.

2. Phân tích thành phần chính (Principal Component Analysis – PCA)

Trong phân tích dữ liệu, PCA là kỹ thuật **giảm chiều** rất phổ biến. Cho một tập điểm dữ liệu, PCA tìm một dãy **hướng vuông góc** sao cho **phương sai** của các điểm theo các hướng đó là lớn nhất. Các hướng được biểu diễn bằng vector đơn vị, gọi là **hệ số thành phần chính** (principal component loadings / coefficients). Chúng tạo thành một hệ tọa độ mới, và ta ánh xạ các điểm vào đó bằng một phép biến đổi tuyến tính trực giao.

Chú giải – Phương sai: đo mức “tán mát” của dữ liệu quanh giá trị trung bình (học ở lớp 10). Phương sai lớn = dữ liệu trải rộng = chứa nhiều thông tin để phân biệt các điểm.

Xét ma trận từ–tài liệu $\mathbf{M} \in \mathbb{R}^{(|V| \times |D|)}$, mỗi hàng là một vector từ $|D|$ chiều. PCA định nghĩa ánh xạ tuyến tính từ $\mathbb{R}^{|D|}$ sang $\mathbb{R}^p$:

$$\mathbf{N} = \mathbf{M} \mathbf{C} \quad (3.35) \quad \mathbf{C} = \begin{bmatrix} \mathbf{c}_1 \\ \vdots \\ \mathbf{c}_p \end{bmatrix} \quad (3.36)$$

trong đó $\mathbf{N} \in \mathbb{R}^{(|V| \times p)}$ là ma trận các vector từ đã ánh xạ, $\mathbf{C} \in \mathbb{R}^{(|D| \times p)}$ là ma trận ánh xạ. Mỗi cột $\mathbf{c}_i = [c_i(1), \dots, c_i(|D|)]^T$ là một nhóm hệ số thành phần chính, biểu thị một hàm tuyến tính kết hợp các đặc trưng đầu vào thành đặc trưng mới. Coi $\mathbf{M}$ là giá trị của các hàm đặc trưng (các cột $\{m_1, \dots, m_{|D|}\}$), ta có:

$$\mathbf{M}\mathbf{c}_i = \begin{bmatrix} \mathbf{m}_1 \\ \vdots \\ \mathbf{m}_{|D|} \end{bmatrix} \mathbf{c}_i = \sum_{k=1}^{|D|} c_i(k) \mathbf{m}_k \quad (3.37)$$

Tích $\mathbf{M}\mathbf{c}_i$ (cột thứ i của $\mathbf{N}$) cho giá trị đặc trưng mới của mọi từ trong $\mathbf{V}$. Các vector $\{\mathbf{c}_1, \dots, \mathbf{c}_p\}$ được sinh **lần lượt** để làm cực đại phương sai của dữ liệu chiếu:

$$\hat{\mathbf{c}}_i = \arg\max_{\mathbf{c}_i} \mathrm{Var}(\mathbf{M}\mathbf{c}_i) = \arg\max_{\mathbf{c}_i} \mathbf{c}_i^T \mathbf{S} \mathbf{c}_i, \quad \mathbf{c}_i^T \mathbf{c}_i = 1 \quad (3.38)$$

trong đó $\mathbf{S}$ là **ma trận hiệp phương sai** $|D| \times |D|$ của $\mathbf{M}$. Để lời giải xác định, ta ràng buộc $\mathbf{c}_i$ là vector đơn vị ($\mathbf{c}_i^T \mathbf{c}_i = 1$). Dùng **nhân tử Lagrange** λ_i , ta có hàm Lagrange:

$$\mathcal{L}(\mathbf{c}_i, \lambda_i) = \mathbf{c}_i^T \mathbf{S} \mathbf{c}_i - \lambda_i (\mathbf{c}_i^T \mathbf{c}_i - 1) \quad (3.39)$$

Giải điểm dừng cho thấy $\hat{\mathbf{c}}_i$ phải là một **vector riêng (eigenvector)** của $\mathbf{S}$, với λ_i là **giá trị riêng (eigenvalue)** tương ứng [Jolliffe, 2002]. Vì $\mathbf{S}$ đối xứng $|D| \times |D|$, nó có đúng $|D|$ giá trị riêng và các vector riêng vuông góc từng đôi. Sắp vector riêng theo giá trị riêng giảm dần, ta lấy p vector đầu làm $\{\hat{\mathbf{c}}_1, \dots, \hat{\mathbf{c}}_p\}$: $\hat{\mathbf{c}}_1$ ứng với giá trị riêng lớn nhất, $\hat{\mathbf{c}}_2$ lớn thứ hai, v.v. Hình chiếu $\mathbf{M}\hat{\mathbf{c}}_i$ gọi là **thành phần chính thứ i** của $\mathbf{M}$.

Chú giải – Vector riêng: Ta muốn tìm hướng làm phương sai lớn nhất, với điều kiện vector chỉ hướng dài 1. Toán học cho thấy các hướng tốt nhất chính là các **vector riêng** của ma trận hiệp phương sai – những hướng mà khi nhân ma trận vào, vector chỉ bị kéo dài/co lại chứ không đổi hướng. Giá trị riêng càng lớn, hướng đó càng chứa nhiều phương sai (thông tin). Nhân tử Lagrange là kỹ thuật giải tích đại học để tìm cực trị khi có ràng buộc.

Có thể hình dung PCA như **đổi hệ tọa độ** trong không gian Euclid. Mỗi hàng của $\mathbf{M}$ là tọa độ một điểm trong không gian $|D|$ chiều $\mathbf{A}$. PCA biểu diễn các điểm này trong hệ tọa độ mới p chiều $\mathbf{B}$, với chiều thứ i theo hướng vector đơn vị $\mathbf{c}_i$. Tọa độ thứ i của một điểm trong $\mathbf{B}$ là hình chiếu của nó lên đường thẳng theo hướng $\mathbf{c}_i$. Hướng $\mathbf{c}_i$ được chọn để các điểm chiếu **trải rộng nhất** – tức tìm trục dọc theo đó dữ liệu tản mát nhất. Hình 3.5 minh họa việc chiếu dữ liệu 2 chiều xuống 1 chiều.

Hình 3.5 (mô tả): Một tập điểm (chấm đen) trên mặt phẳng tạo thành dải xiên từ dưới trái lên trên phải. PCA tìm một hướng (mũi tên “Hướng 1”, chạy dọc dải điểm) sao cho phương sai của các điểm chiếu lên hướng đó (chấm màu) là lớn nhất; “Hướng 2” vuông góc với nó. Hướng 1 tạo thành hệ tọa độ 1 chiều; dữ liệu ánh xạ lên đó gọi là thành phần chính của các điểm ban đầu.

Chú giải – Ví dụ đời thường: Bạn có chiều cao và cân nặng của 100 học sinh (2 chiều). Hai đại lượng tương quan mạnh: người cao thường nặng hơn, nên trên đồ thị các điểm nằm thành dải xiên. PCA tìm ra trục chạy dọc dải đó – tạm gọi là trục “cỡ người”. Chỉ một con số trên trục này đã mô tả được phần lớn khác biệt giữa các học sinh.

Trong thực tế, p thường nhỏ hơn nhiều so với $|D|$, nên PCA giảm đáng kể số chiều của biểu diễn từ. PCA rất tổng quát và hữu ích ở nhiều ngành: nếu M là các quan sát trên một tập biến, PCA cô đọng chúng thành một tập nhỏ hơn các biến **không tương quan** (các thành phần chính).

3. Các phương pháp khác

Giảm chiều là bài toán cơ bản trong học máy và đã được mở rộng theo nhiều hướng. Ví dụ, các mô hình word embedding nơ-ron ở Mục 3.4 và 3.5 vốn học trực tiếp vector từ ít chiều, số thực từ văn bản. Một số phương pháp khác hay gặp:

- **Mô hình chủ đề (Topic models).** Về kỹ thuật, đây không phải cách giảm chiều mà là công cụ mô tả cách tài liệu và từ được sinh ra dựa trên phân phối theo các chủ đề [Blei, 2012]. Ví dụ, **phân bố Dirichlet ẩn (Latent Dirichlet Allocation – LDA)** mô hình hóa việc sinh tài liệu bằng phân phối tài liệu–chủ đề và chủ đề–từ [Blei và cộng sự, 2003]. Như sản phẩm phụ, mỗi chủ đề có một phân phối trên các từ. Viết mọi phân phối chủ đề–từ thành ma trận $|V| \times K$ (K là số chủ đề), ta có biểu diễn từ tương tự các mục trước. Thường K khá nhỏ (ví dụ 200), nên đây là mô hình ít chiều. LDA không phổ biến lắm để học biểu diễn từ, nhưng cho một cách biểu diễn từ như phân phối trên các chủ đề ẩn.
- **Bộ tự mã hóa (Auto-encoders).** Bộ tự mã hóa *undercomplete* là mô hình nơ-ron mã hóa đặc trưng thành **mã ít chiều** sao cho khôi phục lại được đầu vào từ mã đó. Ưu điểm là không giả định gì về cấu trúc ẩn của đặc trưng, nên học được biểu diễn ít chiều cho mọi loại dữ liệu. Chương 7 có ví dụ dùng chúng để học biểu diễn câu; chi tiết xem Chương 2.
- **Giảm chiều có giám sát.** Các phương pháp truyền thống (như PCA) hoạt động **không giám sát**. Khi có nhãn của bài toán đích, tận dụng tín hiệu này sẽ có lợi. Ví dụ, trong **biệt thức tuyến tính Fisher** và **phân tích biệt thức tuyến tính (Linear Discriminant**

Analysis), dữ liệu nhiều chiều được ánh xạ vào không gian ít chiều sao cho các lớp định trước tách xa nhau nhất. Khái niệm này được tổng quát cho không gian đích nhiều chiều qua phân tích biến chính tắc (Canonical Variates analysis) [Barber, 2012].

- **Chọn đặc trưng (Feature selection).** Là xác định tập con các đặc trưng liên quan nhất để biểu diễn đối tượng, qua đó giảm chiều một cách tường minh [Guyon và Elisseeff, 2003; Liu và Motoda, 2012]. Đơn giản nhất là coi nó là bài toán tìm kiếm trong không gian các tập con đặc trưng để làm cực đại (hay cực tiểu) một mục tiêu. Mục tiêu thường phụ thuộc vào bài toán dùng đặc trưng, nên phải cân bằng nhiều yếu tố: hiệu năng bài toán đích, hiệu quả tìm kiếm, cách biểu diễn mỗi tập con. Chọn đặc trưng thường thuộc học có giám sát; về chọn đặc trưng không giám sát, xem tổng quan của Solorio-Fernández và cộng sự [2020].

Chú giải: Chú ý trùng tên: **LDA** có thể là *Latent Dirichlet Allocation* (mô hình chủ đề) hoặc *Linear Discriminant Analysis* (phân tích biệt thức tuyến tính) – hai phương pháp hoàn toàn khác nhau.

Trong thống kê, nhiều phương pháp khác cũng thuộc khung giảm chiều. **Phân tích nhân tố (factor analysis)** giống PCA ở chỗ cùng tìm ánh xạ tuyến tính sang một số ít biến mới; khác biệt là phân tích nhân tố mô hình hóa phương sai **chung** của các biến, còn PCA làm cực đại phương sai của dữ liệu chiều. **Phân tích thành phần độc lập (ICA)** thì tìm các thành phần **độc lập** có thể tách cộng. Xem thêm các giáo trình học máy và thống kê [McClave và Sincich, 2006; Freedman và cộng sự, 2007; Barber, 2012].

3.4 Rút ra word embedding từ mô hình ngôn ngữ nơ-ron (NLM)

Đếm số lần đồng xuất hiện từ–từ hay từ–tài liệu là cách đơn giản để biểu diễn từ dựa trên đặc điểm phân bố trong kho văn bản. Dù hiệu quả ở nhiều ứng dụng, cách này áp một ràng buộc chặt lên biểu diễn từ: mỗi phần tử của vector từ phải diễn giải được như bằng chứng tường minh về cách từ phân bố trong các ngữ cảnh. Lý tưởng hơn, ta muốn biểu diễn từ ở dạng tổng quát hơn – các **vector số thực dày đặc** (gọi là **word embedding**) – mà không áp đặt ràng buộc hay giả định nào về ý nghĩa của từng phần tử.

Học vector từ không ràng buộc có cái giá của nó. Khác với các phương pháp dựa trên đếm ở Mục 3.3, ta không dựa vào kinh nghiệm hay hiểu biết thống kê có sẵn để ước lượng giá trị vector, mà muốn **suy ra biểu diễn có ý nghĩa trực tiếp từ dữ liệu**. Thách thức chính là không có “đáp án chuẩn”, vì không thể nhờ người gán tay các biểu diễn liên tục nhiều chiều

cho cả từ vựng. Vì vậy, ta thường đặt việc học vector từ như một **mục tiêu phụ** bên trong một bài toán đã được định nghĩa rõ (gọi là **bài toán nền – background task**). Vector từ học được xuất hiện như một **sản phẩm phụ** có giá trị của việc huấn luyện trên bài toán nền.

Chú giải – Ý tưởng chính: Ta không biết “đáp án đúng” cho vector của từ “mèo” là gì. Nhưng ta có thể giao cho máy một bài tập có đáp án rõ ràng, như **đoán từ tiếp theo** trong câu (“Con ... đang bắt chuột” → “mèo”). Để làm tốt bài tập đó, máy buộc phải tự xây dựng các vector từ hợp lý. Làm xong bài tập, ta giữ lại các vector đó dùng cho việc khác. Giống như học sinh luyện giải đề toán, và “sản phẩm phụ” là tư duy logic dùng được cho mọi môn.

Một ví dụ phổ biến là rút vector từ từ **mô hình ngôn ngữ nơ-ron (neural language model – NLM)**. Nhắc lại NLM ở Chương 2: mục tiêu là dự đoán xác suất của một từ khi biết các từ đứng trước [Bengio và cộng sự, 2003]. Cụ thể, gọi w_i là từ cần dự đoán và $\{w_{i-n+1}, \dots, w_{i-1}\}$ là các từ ngữ cảnh đã thấy. Trước hết, các từ ngữ cảnh được chiếu thành vector dày đặc d_e chiều $\{e_{i-n+1}, \dots, e_{i-1}\}$ qua một **lớp embedding**. Giả sử $\mathbf{1}_{w_j}$ là biểu diễn one-hot của từ j (vector hàng kích thước $|V|$), vector dày đặc e_j là:

$$\mathbf{e}_j = \mathbf{1}_{w_j} \mathbf{C} \quad (3.40)$$

trong đó $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$ là tham số của lớp embedding. $\mathbf{C}$ thường gọi là **bảng embedding từ (word embedding table)**, trong đó hàng thứ k là biểu diễn của từ thứ k trong V .

Chú giải – Vì sao nhân one-hot với $\mathbf{C}$ lại ra embedding? Nhân một vector toàn số 0, chỉ có một số 1 ở vị trí k , với ma trận $\mathbf{C}$ thì kết quả đúng bằng **hàng thứ k** của $\mathbf{C}$. Nói cách khác, (3.40) chỉ là cách viết toán học của việc **tra bảng**: muốn lấy vector của từ “mèo” (số thứ tự k), ta lấy hàng k của bảng $\mathbf{C}$. Bảng $\mathbf{C}$ có $|V|$ hàng (mỗi từ một hàng) và d_e cột (ví dụ 300). Ban đầu $\mathbf{C}$ chứa số ngẫu nhiên; trong quá trình huấn luyện, máy chỉnh dần các con số này.

Sau đó, dùng một **mạng nơ-ron truyền thẳng (feed-forward neural network)** để tính phân phối xác suất của từ ở vị trí i :

$$\Pr(\cdot \mid w_{i-n+1}, \dots, w_{i-1}) = F_{\theta}(\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}) \quad (3.41)$$

trong đó $F_{\theta}(\cdot)$ là mạng nơ-ron truyền thẳng có tham số θ . Lớp embedding có thể coi là một thành phần của NLM. Ở đây ta dùng ký hiệu hơi khác để nhấn mạnh NLM là hàm của cả θ và $\mathbf{C}$:

$$\Pr_{\{\theta, \mathbf{C}\}}(\cdot \mid w_{i-n+1}, \dots, w_{i-1}) = F_{\{\theta, \mathbf{C}\}}(w_{i-n+1}, \dots, w_{i-1}) \quad (3.42)$$

Để huấn luyện, ta tối ưu cả θ và C để giảm thiểu một hàm mất mát. Phương pháp phổ biến là **huấn luyện hợp lý cực đại (maximum likelihood training)**, làm cực đại tổng log-likelihood trên mọi n -gram trong dữ liệu. Cho dãy từ $w_1 \dots w_m$, mục tiêu huấn luyện là (có thể mở rộng cho tập dữ liệu nhiều dãy):

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg\max_{\{\theta, \mathbf{C}\}} \sum_{i=1}^m \log \Pr_{\{\theta, \mathbf{C}\}}(w_i \mid w_{i-n+1}, \dots, w_{i-1}) \quad (3.43)$$

Chú giải – Mạng nơ-ron và huấn luyện: Mạng nơ-ron là một hàm toán học lớn có rất nhiều tham số (các con số có thể chỉnh). “Huấn luyện” là quá trình chỉnh các tham số đó dần dần sao cho mô hình đoán đúng hơn trên dữ liệu. Công thức (3.43) nói: hãy chọn θ và C sao cho tổng log-xác suất mô hình gán cho **từ đúng** ở mọi vị trí trong văn bản là lớn nhất – tức mô hình “ít bất ngờ” nhất khi đọc văn bản thật. Việc chỉnh tham số thường dùng phương pháp **hạ gradient** (giống đi xuống dốc từng bước nhỏ để tìm điểm thấp nhất của một thung lũng).

Có được tham số tối ưu $\hat{\theta}$ và $\hat{C}$, ta có thể áp dụng $F_{\hat{\theta}, \hat{C}}(\cdot)$ cho các n -gram chưa gặp. Quan trọng hơn, ta có một bộ vector từ đã huấn luyện tốt (chính là $\hat{C}$) dùng được trong **các hệ thống khác ngoài NLM**. Điều này gọi là **tiền huấn luyện (pre-training)** vector từ. Trong tiền huấn luyện, $F_{\theta, C}(\cdot)$ có thể là bất kỳ hệ thống nào dùng vector từ C . Như vậy, bài toán học C được chuyển thành bài toán tối ưu $F_{\theta, C}(\cdot)$ trên bài toán nền (Hình 3.6). Ưu điểm chính là tận dụng được các bài toán NLP có sẵn để huấn luyện vector từ. Rủi ro là vector từ “tốt nhất” tìm được khi huấn luyện $F_{\theta, C}(\cdot)$ có thể không phù hợp với hệ thống nơi vector từ thực sự được dùng. Thụ vị là trong nhiều trường hợp, vector từ tiền huấn luyện bằng NLM có chất lượng tốt cho các bài toán phía sau, hoặc ít nhất là điểm xuất phát tốt để tinh chỉnh thêm trong hệ thống đích.

Hình 3.6 (mô tả): Minh họa tiền huấn luyện vector từ trong NLM, gồm hai giai đoạn nối tiếp.

Giai đoạn 1 – Huấn luyện vector từ: NLM được viết thành hàm $F_{\theta, C}(\cdot)$ của bảng embedding C và các tham số khác θ ; ta tối ưu đồng thời C và θ qua mô hình ngôn ngữ. Kết quả là bảng vector từ đã tối ưu $\hat{C}$. **Giai đoạn 2 – Áp dụng vector từ:** $\hat{C}$ được đưa sang một hệ thống mới $G_{\pi, \hat{C}}(\cdot)$ cho bài toán mới (có thể khác NLM), trong đó $\hat{C}$ là bảng embedding học từ NLM và π là các tham số riêng của $G(\cdot)$.

Chú giải – Tiên huấn luyện: Đây là ý tưởng cực kỳ quan trọng của AI hiện đại. Thay vì mỗi bài toán (dịch máy, phân loại cảm xúc, hỏi đáp...) lại học từ đầu, ta **học trước** một lần trên kho văn bản khổng lồ, rồi mang kiến thức đó sang các bài toán khác. Các mô hình ngôn ngữ lớn như GPT hay Claude là phiên bản phát triển vượt bậc của chính ý tưởng này.

3.5 Các mô hình word embedding

Về nguyên tắc, vector từ có thể học theo bất kỳ cách nào. Coi vector từ là thành phần của các hệ thống NLP có sẵn là một lựa chọn, nhưng cách này thường thiếu cân nhắc riêng cho bài toán biểu diễn. Lựa chọn khác là phát triển phương pháp **thiết kế riêng** cho việc học biểu diễn. Vì việc huấn luyện các hệ thống này không phải thỏa mãn các ràng buộc chặt của bài toán NLP tiêu chuẩn, việc thu được vector từ chất lượng cao trở nên dễ hơn nhiều.

3.5.1 Word2Vec

Word2Vec là một bộ mô hình được đề xuất trong [Mikolov và cộng sự, 2013a; c]. Giống mô hình ngôn ngữ nơ-ron, Word2Vec dựa trên mạng nơ-ron. Tuy nhiên, thay vì dùng mô hình sinh tự hồi quy nặng nề cho n-gram, Word2Vec đặt việc học vector từ theo kiểu **log-tuyến tính (log-linear)** hiệu quả về tính toán. Vì vậy, kiến trúc của nó khác hẳn kiến trúc mô hình ngôn ngữ truyền thống. Họ Word2Vec có hai kiến trúc chính:

Mô hình túi từ liên tục (continuous bag-of-words – CBOW). CBOW là mô hình dự đoán từ: dự đoán khả năng một từ xuất hiện ở vị trí i khi biết cửa sổ $-n$ và $+n$ từ quanh nó. Cấu trúc CBOW giống mô hình ngôn ngữ nơ-ron ở Chương 2 (Hình 3.7a). Trước hết, dùng lớp embedding biến các từ ngữ cảnh $w_{i-n}, \dots, w_{i-1}$ và $w_{i+1}, \dots, w_{i+n}$ thành vector từ, bằng cách nhân biểu diễn one-hot của mỗi từ w_j với bảng embedding $C \in \mathbb{R}^{(|V| \times d_e)}$ như (3.40). Các vector này được **lấy trung bình** để có một biểu diễn duy nhất cho các từ đầu vào:

$$\mathbf{h} = \frac{1}{2n} \left(\sum_{j=i-n}^{i-1} \mathbf{1}_{w_j} \mathbf{C} + \sum_{j=i+1}^{i+n} \mathbf{1}_{w_j} \mathbf{C} \right) \quad (3.44)$$

Lưu ý mô hình này hoàn toàn **bỏ qua thứ tự** từ đầu vào vì dùng phép cộng. Đó là lý do nó được gọi là *túi từ*. Lớp đầu ra của CBOW là lớp **Softmax** tiêu chuẩn chiều h thành phân phối xác suất trên từ vựng:

$$\mathbf{y} = \text{Softmax}(\mathbf{h} \mathbf{U} + \mathbf{b}) \quad (3.45)$$

trong đó $U \in \mathbb{R}^{(d_e \times |V|)}$ là ma trận tham số của ánh xạ tuyến tính và $b \in \mathbb{R}^{|V|}$ là hệ số tự do (bias). y là phân phối trên từ vựng, và $\Pr(w_i | w_{i-n}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+n}) = y(w_i)$. Các công thức (3.44)–(3.45) mô tả một mạng nơ-ron rất đơn giản. Ưu điểm là mô hình nhỏ và hiệu quả so với NLN. Huấn luyện CBOW rất thẳng: tìm ước lượng hợp lý cực đại của tham số. Gọi θ là các tham số ngoài C ($\theta = \{U, b\}$):

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg\max_{\theta, \mathbf{C}} \sum_{i=n+1}^{m-n-1} \log \Pr_{\theta, \mathbf{C}}(w_i \mid w_{i-n}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+n}) \quad (3.46)$$

trong đó m là độ dài dãy từ. Sau huấn luyện, ta bỏ $\hat{\theta}$ và chỉ dùng $\hat{\mathbf{C}}$ làm bảng tra vector từ.

Chú giải – Softmax: là hàm biến một dãy số bất kỳ thành một dãy xác suất (không âm, tổng bằng 1). Công thức: $\text{softmax}(z_k) = e^{z_k} / \sum_j e^{z_j}$. Ví dụ điểm (2, 1, 0) $\rightarrow$ xác suất $\approx (0,67; 0,24; 0,09)$. Số càng lớn thì xác suất càng cao. **CBOW giống bài tập điền vào chỗ trống:** “Tôi uống một ly ___ sữa đá” $\rightarrow$ nhìn các từ xung quanh để đoán “cà phê”.

Mô hình skip-gram liên tục (continuous skip-gram). Skip-gram cũng là mô hình dự đoán từ, nhưng làm **ngược lại** CBOW: dự đoán từng từ trong $\pm n$ từ ngữ cảnh khi biết w_i . Bài toán thường được đặt thành ước lượng xác suất w_j xuất hiện khi biết w_i ($i-n \leq j \leq i-1$ hoặc $i+1 \leq j \leq i+n$). Hình 3.7b cho cấu trúc skip-gram. Lớp embedding xử lý w_i như thường:

$$\mathbf{h} = \mathbf{h}_{w_i} \in \mathbf{C} \quad (3.47)$$

Rồi đưa qua lớp Softmax để dự đoán xác suất cho mỗi từ ngữ cảnh w_j (giả sử $j = i + k$; $k > 0$ là từ ở cửa sổ bên phải, $k < 0$ là bên trái):

$$\mathbf{y}_k = \text{Softmax}(\mathbf{h} \cdot \mathbf{V}_k + \mathbf{b}_k) \quad (3.48)$$

trong đó $\mathbf{V}_k$ và $\mathbf{b}_k$ là tham số mô hình ($-n \leq k \leq -1$ và $1 \leq k \leq n$). Ta có:

$$\Pr(w_j \mid w_i) = \Pr(w_{i+k} \mid w_i) = \mathbf{y}_k(w_{i+k}) \quad (3.49)$$

Gọi θ là viết tắt của $\{\mathbf{V}_k\}$ và $\{\mathbf{b}_k\}$. Bài toán huấn luyện là:

$$(\hat{\theta}, \hat{\mathbf{C}}) = \arg\max_{\theta, \mathbf{C}} \sum_{i=n+1}^{m-n-1} \sum_{-n \leq k \leq -1, 1 \leq k \leq n} \log \Pr_{\theta, \mathbf{C}}(w_{i+k} \mid w_i) \quad (3.50)$$

Chú giải – Skip-gram: ngược với CBOW: cho một từ ở giữa, đoán các từ xung quanh. Cho “cà phê” → đoán các từ hay đứng gần như “uống”, “ly”, “sữa”, “đá”. Hai từ có “hàng xóm” giống nhau (“cà phê” và “trà”) sẽ buộc phải có vector giống nhau để cùng đoán tốt.

Cả hai mô hình đều giống **bài kiểm tra điền từ (cloze test)** khi chỉ xét quan hệ từng cặp giữa các từ. Vì bỏ qua thứ tự từ và quan hệ cú pháp phức tạp, phân phối dự đoán của chúng kém chính xác hơn mô hình ngôn ngữ. Tuy nhiên, mục tiêu của các mô hình này không phải là đoán chính xác từ bị thiếu, mà là **học biểu diễn từ** từ một bài toán nắm bắt được quan hệ từ-từ. Vì vậy, hiệu năng đoán từ của mô hình không quá quan trọng.

Một ưu điểm khác là kiến trúc **rất đơn giản, dễ huấn luyện**. Ví dụ, cả hai mô hình đều không có lớp ẩn, lớp embedding nổi thẳng vào lớp đầu ra. Cấu trúc này là ví dụ của **mô hình log-tuyến tính** trong học máy: các biến đầu vào được biến đổi tuyến tính thành một vector đặc trưng dày đặc (ví dụ (3.44)), theo sau ngay là một hàm phân loại log-tuyến tính (ví dụ (3.45)).

*Hình 3.7 (mô tả): Kiến trúc CBOW và skip-gram với cửa sổ ± 2 . (a) **CBOW:** bốn từ ngữ cảnh $w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}$ đi qua lớp embedding thành $e_{i-2} = \mathbf{1}_{w_{i-2}}C, \dots, e_{i+2} = \mathbf{1}_{w_{i+2}}C$; lấy trung bình $h = \frac{1}{4}(e_{i-2} + e_{i-1} + e_{i+1} + e_{i+2})$; rồi $y = \text{Softmax}(hU + b)$; xác suất dự đoán $\Pr(w_i | w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}) = y(w_i)$. (b) **Skip-gram:** một từ w_i đi qua lớp embedding thành $h = \mathbf{1}_{w_i}C$; từ h tỏa ra bốn lớp Softmax $y_k = \text{Softmax}(hV_k + b_k)$, $k \in \{-2, -1, 1, 2\}$, mỗi lớp cho $\Pr(w_{i+k} | w_i) = y_k(w_{i+k})$. Cả hai được huấn luyện theo hợp lý cực đại; bảng tra của lớp embedding chính là vector từ cho các từ trong từ vựng.*

Chú giải: Word2Vec (2013) tạo ra bước ngoặt lớn vì huấn luyện được trên hàng tỷ từ chỉ trong vài giờ, và cho ra các vector có tính chất kỳ diệu như “vua – đàn ông + phụ nữ $\approx$ nữ hoàng” (xem Mục 3.6.2).

3.5.2 GloVe

Vector toàn cục (Global Vectors – GloVe) là word embedding học bằng cách kết hợp ưu điểm của **phân tích ma trận toàn cục** và **phương pháp cửa sổ ngữ cảnh cục bộ** [Pennington và cộng sự, 2014]. GloVe bắt đầu bằng việc xây ma trận đồng xuất hiện từ-từ toàn cục (như Mục 3.3), rồi xây một mô hình **log-song tuyến tính (log-bilinear)** dựa trên một số giả định lý thuyết về nghĩa của từ.

Cho ma trận đồng xuất hiện M , mỗi ô $M(a, b) = \text{count}(a, b)$ là số lần từ $a \in V$ cùng xuất hiện với từ $b \in V$ trong một cửa sổ ngữ cảnh, ta tính xác suất có điều kiện $\Pr(b | a)$:

$$\Pr(b \mid a) = \frac{\mathrm{count}(a, b)}{\sum_{b'} \mathrm{count}(a, b')} \\ = \frac{\mathrm{count}(a, b)}{\mathrm{count}(a)} \quad (3.51)$$

trong đó $\mathrm{count}(a)$ là tổng số lần từ a xuất hiện trong kho văn bản.

Để hiểu động cơ của GloVe, xét ví dụ phân biệt hai khái niệm *air* (không khí) và *water* (nước). Dùng (3.51), ta tính xác suất các từ đích này xuất hiện khi biết các **từ thăm dò (probe words)**. Bảng sau là một phần nhỏ của ma trận $\Pr(b|a)$ tính từ kho 3,8 tỷ từ của WMT14:

Đại lượng	w = fly (bay)	w = drink (uống)	w = breath (hơi thở)	w = live (sống)	w = flow (chảy)
$\Pr(\text{air} \mid w)$	$1,5 \times 10^{-4}$	$6,2 \times 10^{-5}$	$2,2 \times 10^{-4}$	$1,6 \times 10^{-4}$	$3,6 \times 10^{-4}$
$\Pr(\text{water} \mid w)$	$1,3 \times 10^{-5}$	$4,1 \times 10^{-4}$	$1,8 \times 10^{-5}$	$1,4 \times 10^{-4}$	$3,0 \times 10^{-4}$
$\Pr(\text{air} \mid w) / \Pr(\text{water} \mid w)$	11,54	0,15	12,2	1,14	1,2

Trong bảng, $\Pr(\text{air}|w)$ và $\Pr(\text{water}|w)$ cho biết *air* và *water* tương quan mạnh thế nào với các từ thăm dò w . Phát hiện then chốt nằm ở **tỉ số xác suất** $\Pr(\text{air}|w)/\Pr(\text{water}|w)$ ở hàng cuối. Tỉ số này tách bạch được **sự khác biệt ngữ nghĩa** giữa hai từ đích. Khi w liên quan nhiều tới *air* mà không liên quan *water* ($w = \text{fly}$ hay *breath*), tỉ số lớn. Ngược lại, khi w liên quan tới *water* mà không liên quan *air* ($w = \text{drink}$), tỉ số nhỏ. Khi w liên quan tới cả hai ($w = \text{live}$, *flow*) hoặc không liên quan cả hai, tỉ số gần 1.

Chú giải – Vì sao dùng tỉ số? Nhìn riêng một xác suất thì khó hiểu: $\Pr(\text{air} \mid \text{flow}) = 3,6 \times 10^{-4}$ là cao hay thấp? Nhưng khi chia cho $\Pr(\text{water} \mid \text{flow})$, ta được $1,2 \approx 1$, tức “chảy” liên quan tới cả hai như nhau (khí và nước đều chảy). Còn “uống” cho tỉ số 0,15 – rõ ràng nghiêng về nước. Tỉ số giúp “khử” những yếu tố chung và làm nổi bật điểm khác biệt.

Tiền đề nền tảng của GloVe là vector từ nên **mã hóa trực tiếp tỉ số xác suất đồng xuất hiện** này. Điểm xuất phát tự nhiên là giả định có một hàm F nhận vector từ làm đầu vào và xấp xỉ tỉ số:

$$F(\mathbf{e}_a, \mathbf{e}_b, \tilde{\mathbf{e}}_w) = \frac{\Pr(a \mid w)}{\Pr(b \mid w)} \quad (3.52)$$

trong đó $e_a, e_b \in \mathbb{R}^{d_e}$ là vector **đích (target)** của từ a và b , còn $\tilde{e}_w \in \mathbb{R}^{d_e}$ là vector **ngữ cảnh (context)** của từ thăm dò w . Chú ý sự phân biệt giữa e và $\tilde{e}$: e lấy từ bảng embedding đích C , còn $\tilde{e}$ lấy từ một bảng embedding ngữ cảnh riêng $\tilde{C}$. Dùng hai bảng riêng giúp giảm **quá khớp (overfitting)** khi huấn luyện. Cuối cùng, biểu diễn từ thường lấy là tổng (hoặc trung bình) của hai bảng: $e_{\text{final}} = C + \tilde{C}$.

Vì không gian vector vốn tuyến tính, cách tự nhiên nhất để mã hóa quan hệ giữa hai từ đích a và b là **phép trừ vector**: $e_a - e_b$. Hơn nữa, để đo quan hệ giữa hiệu này và từ ngữ cảnh w , ta dùng **tích vô hướng**. Điều này giới hạn hàm F chỉ nhận một đầu vào vô hướng:

$$F\big(\langle \mathbf{e}_a - \mathbf{e}_b, \tilde{\mathbf{e}}_w \rangle\big) = \frac{\Pr(a \mid w)}{\Pr(b \mid w)} \quad (3.53)$$

Để có cấu trúc đại số chặt chẽ, ta yêu cầu F là một **đồng cấu (homomorphism)** giữa nhóm cộng trong không gian vector và nhóm nhân trong không gian xác suất. Điều này bảo đảm phép trừ vector ứng đúng với phép chia xác suất:

$$F\big(\langle \mathbf{e}_a - \mathbf{e}_b, \tilde{\mathbf{e}}_w \rangle\big) = \frac{F(\langle \mathbf{e}_a, \tilde{\mathbf{e}}_w \rangle)}{F(\langle \mathbf{e}_b, \tilde{\mathbf{e}}_w \rangle)} \quad (3.54)$$

Nghiệm liên tục duy nhất của (3.54) là **hàm mũ** $F(x) = \exp(x)$. Do đó:

$$F(\langle \mathbf{e}_a, \tilde{\mathbf{e}}_w \rangle) = \exp(\langle \mathbf{e}_a, \tilde{\mathbf{e}}_w \rangle) = \Pr(a \mid w) = \frac{\text{count}(a, w)}{\text{count}(a)} \quad (3.55)$$

Chú giải – Vì sao là hàm mũ? Ta cần hàm F biến “hiệu” thành “thương”: $F(x - y) = F(x) / F(y)$. Bạn đã học ở lớp 12 tính chất $e^{(x-y)} = e^x / e^y$ – hàm mũ làm đúng điều này. (Chi tiết nhỏ: theo định nghĩa (3.51), $\text{count}(a, w) / \text{count}(a)$ là $\Pr(w|a)$; vì ma trận đồng xuất hiện đối xứng nên $\text{count}(a, w) = \text{count}(w, a)$. Sự bất đối xứng này được xử lý ở bước tiếp theo bằng các hệ số bias.)

Lấy logarit hai vế:

$$\langle \mathbf{e}_a, \tilde{\mathbf{e}}_w \rangle = \log \text{count}(a, w) - \log \text{count}(a) \quad (3.56)$$

Vấn đề của (3.56) là số hạng $\log \text{count}(a)$ **phá vỡ tính đối xứng**: đổi chỗ từ đích a và từ ngữ cảnh w (tương ứng chuyển vị ma trận đồng xuất hiện) lẽ ra không làm thay đổi cấu trúc hình học. Để khôi phục tính đối xứng, số hạng $\log \text{count}(a)$ được thay bằng hai hệ số tự do học được β_a và $\tilde{\beta}_w$:

$$\mathbf{e}_a \tilde{\mathbf{e}}_w^{\top} + \beta_a + \tilde{\beta}_w = \log \text{count}(a, w) \quad (3.57)$$

Phương trình (3.57) là quan hệ lý tưởng ta muốn các embedding thỏa mãn. Để ép điều này, ta lập một **bài toán hồi quy**: giảm thiểu bình phương sai lệch giữa log đồng xuất hiện dự đoán và thực tế. Mật mát bình phương cho một cặp từ (a, w) là:

$$\mathcal{L}_{a,w} = \big(\mathbf{e}_a \tilde{\mathbf{e}}_w^{\top} + \beta_a + \tilde{\beta}_w - \log \text{count}(a, w) \big)^2 \quad (3.58)$$

Hàm chi phí tổng trên toàn từ vựng là tổng có trọng số của các mật mát riêng lẻ:

$$\mathcal{L}_{\text{GloVe}} = \sum_{a, w \in V} f(\text{count}(a, w)) \cdot \mathcal{L}_{a,w} \quad (3.59)$$

trong đó $f(\text{count}(a, w))$ là **hàm trọng số**, được thiết kế để các cặp từ hiếm không lẫn át mất mát, đồng thời giới hạn ảnh hưởng của các cặp quá phổ biến. Pennington và cộng sự [2014] đề xuất:

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}} \right)^{\alpha} & \text{nếu } x < x_{\max} \\ 1 & \text{ngược lại} \end{cases} \quad (3.60)$$

với $x_{\max}$ và α là siêu tham số (thường $x_{\max} = 100$, $\alpha = 3/4$). Như vậy, $f(x)$ phạt các cặp hiếm bằng trọng số nhỏ, đồng thời bảo đảm trọng số của các cặp phổ biến không tăng vô hạn.

Chú giải – Hàm trọng số bằng số: Một cặp từ chỉ gặp 1 lần: $f(1) = (1/100)^{0,75} \approx 0,03$ – gần như không được tính (có thể chỉ là ngẫu nhiên). Cặp gặp 50 lần: $f(50) = 0,5^{0,75} \approx 0,59$. Cặp gặp 10.000 lần (như “của” với “và”): $f = 1$, không hơn – tránh để các từ quá phổ biến chiếm hết sự chú ý. Ngoài ra, cặp có $\text{count} = 0$ có $f(0) = 0$, nên ta tránh được $\log 0$ (không xác định).

Các công thức (3.58)–(3.59) mô tả một khung rất hiệu quả để học vector từ bằng các phép toán mạng nơ-ron tiêu chuẩn (tích vô hướng, cộng vô hướng). Đặc điểm nổi bật của GloVe là dựa trên một mục tiêu **log-song tuyến tính đơn giản**. Vì việc huấn luyện tránh được hàm

mất mát entropy chéo và chuẩn hóa softmax tốn kém mà NLM đầy đủ và Word2Vec cần, GloVe **huấn luyện rất nhanh**. Trực giác cốt lõi là quan hệ ngữ nghĩa có thể mô hình hóa bằng **hồi quy đơn giản trên thống kê toàn cục của kho văn bản**, thay vì chỉ dựa vào độ phân kỳ xác suất trên các cửa sổ ngữ cảnh cục bộ.

Một điểm cần nhắc lại về dữ liệu toàn cục. Ma trận đồng xuất hiện chứa thông tin **gộp từ toàn bộ kho văn bản**. Bằng cách phân tích trực tiếp ma trận này, GloVe đặt việc học thành một bài toán **tối ưu toàn cục** tường minh. Word2Vec cũng tìm bộ tham số tối ưu toàn cục (làm cực đại likelihood trên toàn bộ dữ liệu), nhưng mục tiêu của nó phức tạp và thường được tối ưu bằng học trực tuyến (hạ gradient ngẫu nhiên trên từng lô dữ liệu cục bộ). GloVe, nhờ gộp sẵn dữ liệu vào ma trận đồng xuất hiện, chắt lọc quá trình học thành một mô hình toàn cục trực tiếp và hiệu quả hơn.

Chú giải – So sánh nhanh: Word2Vec giống học sinh đọc từng câu một và rút kinh nghiệm dần. GloVe giống học sinh lập trước một bảng thống kê tổng hợp của cả cuốn sách, rồi học từ bảng đó.

3.5.3 Nhận xét

Các mục trước đã cho thấy vector từ có thể được rút ra bằng nhiều phương pháp. Giờ ta bàn một số cân nhắc quan trọng khi huấn luyện hay áp dụng word embedding trong thực tế.

1. Dựa trên đếm hay dựa trên mạng nơ-ron

Tính đơn giản và dễ diễn giải của các phương pháp dựa trên đếm từ lâu đã được cộng đồng NLP đánh giá cao. Cả hai loại mô hình về cơ bản đều dựa trên **giả thuyết phân bố** – từ xuất hiện trong ngữ cảnh giống nhau thì có nghĩa giống nhau. Tuy nhiên, phương pháp dựa trên đếm xây dựng tường minh các ma trận **thừa, nhiều chiều**, thường dẫn tới vấn đề dữ liệu thừa do **lời nguyền số chiều (curse of dimensionality)**.

Ở đầu bên kia là các mô hình dựa trên mạng nơ-ron. Thay vì đếm đồng xuất hiện và đặt trọng số thủ công, các phương pháp này coi vector từ là **trạng thái trung gian ít chiều, dày đặc** trong một mạng nơ-ron được tối ưu cho một bài toán nền cụ thể. Nhờ bỏ đi các ràng buộc ngôn ngữ học tường minh về ý nghĩa của từng chiều vector, mô hình nơ-ron học được các đặc trưng ngữ nghĩa ẩn mà con người khó tự thiết kế.

Sự tương phản giữa hai cách tiếp cận giống với so sánh rộng hơn giữa hai trường phái học máy nổi tiếng: **kỹ nghệ đặc trưng (feature engineering)** và **học biểu diễn đầu-cuối (end-to-end representation learning)**. Ta không đi sâu vào tranh luận này. Chỉ cần lưu ý rằng

không có dạng nào hơn tuyệt đối. Nói chung, chọn loại vector từ nào phụ thuộc hoàn toàn vào ứng dụng đích và mức độ dễ diễn giải mong muốn. Ví dụ, nếu mục tiêu là một dạng tính toán dễ diễn giải, minh bạch về toán học lấy trực tiếp từ kho văn bản, thì phân tích ma trận đồng xuất hiện vẫn là lựa chọn vững chắc. Ngược lại, nếu cần vector từ liên tục, dày đặc để tích hợp trơn tru vào một kiến trúc nơ-ron sâu, học biểu diễn nơ-ron thường được ưu tiên.

Cần lưu ý rằng học word embedding liên tục, dày đặc đã trở thành **hướng chủ đạo** trong những năm gần đây, mang lại tiến bộ lớn cho các mô hình nơ-ron trong NLP. Song song, giới học thuật cũng rất quan tâm tới việc so sánh hai loại phương pháp và chỉ ra mối liên hệ lý thuyết sâu sắc giữa chúng [Levy và Goldberg, 2014b; Baroni và cộng sự, 2014; Levy và Goldberg, 2014c; Schnabel và cộng sự, 2015a; Levy và cộng sự, 2015; Gladkova và cộng sự, 2016].

Chú giải: Kỹ nghệ đặc trưng = con người tự nghĩ ra các con số để mô tả dữ liệu (như đếm, TF-IDF). **Học đầu-cuối** = để máy tự tìm ra các con số phù hợp nhất từ dữ liệu. **Lời nguyên số chiều**: khi số chiều quá lớn, dữ liệu trở nên “loãng” – hầu hết các ô đều trống, khiến mô hình khó rút ra quy luật. Một kết quả lý thuyết thú vị (Levy và Goldberg, 2014c): skip-gram thực chất đang ngầm phân tích một ma trận PMI – tức hai trường phái gần nhau hơn ta tưởng.

2. Mô hình nông và mô hình sâu

Mạng nơ-ron đã thành cách chuẩn để học vector từ, nhưng các cấu trúc mô hình giới thiệu đến giờ đều đơn giản. Về kỹ thuật, chúng chỉ có một hoặc hai lớp nơ-ron nên được xếp vào loại **mô hình nông (shallow models)**. Ví dụ đáng chú ý là mô hình vLBL [Mnih và Kavukcuoglu, 2013], mô hình hóa tương tác giữa các từ bằng mạng nơ-ron chỉ hai lớp. Kiến trúc này – bỏ cả hàm chuẩn hóa Softmax – là một trong những mô hình word embedding đơn giản nhất theo hiểu biết của tác giả. Dù đơn giản, các mô hình này vẫn rất hiệu quả trong nhiều tình huống. Lợi ích chính của mô hình nông là **hiệu quả tính toán** và **khả năng mở rộng** cho dữ liệu khổng lồ, nhờ đó áp dụng được cho các bài toán NLP phức tạp hơn. Ví dụ nổi bật là hệ thống **fastText** cho phân loại văn bản [Joulin và cộng sự, 2017], có kiến trúc giống CBOW (Mục 3.5.1): văn bản được biểu diễn bằng túi các vector từ, lấy trung bình thành một biểu diễn ẩn chung, theo sau ngay là lớp đầu ra ánh xạ thành phân phối xác suất trên các lớp định trước. Theo cách này, mục tiêu phân loại và vector từ được huấn luyện cùng lúc.

Dù mô hình nông rất hiệu quả để học biểu diễn từ tĩnh, **mạng nơ-ron sâu** có năng lực biểu diễn lớn hơn nhiều. Như hầu hết kiến trúc nhiều lớp, học biểu diễn bằng mạng sâu có một số ưu điểm rõ rệt [Telgarsky, 2016]. Thứ nhất, mô hình sâu cho phép khám phá **không gian**

giả thuyết lớn hơn nhiều, có thể tìm được ánh xạ tốt hơn. Thứ hai, nhiều phép biến đổi **phi tuyến** chồng lên nhau làm tăng mạnh khả năng nắm bắt các mẫu rất phức tạp trong dữ liệu. Có nhiều chiến lược tận dụng độ sâu; đơn giản nhất là chồng dân các lớp truyền thẳng, hồi quy hay tích chập lên trên lớp embedding. Với sự phổ biến của các mô hình tiền huấn luyện quy mô lớn trong NLP, xu hướng đã chuyển sang lấy **biểu diễn theo ngữ cảnh** trực tiếp từ các kiến trúc rất sâu, đạt kết quả tốt nhất trên rất nhiều bộ đánh giá NLP [Radford và cộng sự, 2018; Devlin và cộng sự, 2019].

Tuy nhiên, năng lực mô hình hóa tăng lên kéo theo **nút thắt tính toán** nghiêm trọng và **huấn luyện thiếu ổn định**. Vì vậy, rất nhiều nghiên cứu tập trung giải quyết các thách thức tối ưu này [Pascanu và cộng sự, 2013; Bapna và cộng sự, 2018; Wang và cộng sự, 2019; Zhang và cộng sự, 2019a; Pham và cộng sự, 2019; Li và cộng sự, 2020]. Chương 4–6 sẽ giới thiệu một số mô hình NLP thành công xây dựng trên các kiến trúc nơ-ron sâu này.

Chú giải: “Nông” và “sâu” chỉ số **lớp** xử lý chồng lên nhau trong mạng nơ-ron. Mạng nông giống một dây chuyền 1–2 công đoạn: nhanh, rẻ. Mạng sâu có hàng chục đến hàng trăm công đoạn: hiểu được những điều tinh tế hơn nhưng tốn nhiều máy tính và khó huấn luyện hơn. **Phi tuyến** nghĩa là quan hệ không phải đường thẳng – cần thiết để mô tả các hiện tượng phức tạp của ngôn ngữ. Các mô hình như BERT và GPT (trích dẫn Devlin 2019, Radford 2018) chính là mạng sâu.

3. Mục tiêu huấn luyện

Ý tưởng coi vector từ là tham số mô hình phù hợp với **mô hình hóa biến ẩn**: mô hình có tham số là các vector từ học được, và giá trị tối ưu của chúng được suy ra bằng cách tối ưu một hàm mục tiêu toàn cục. Dù khuôn mẫu này là chuẩn, mục tiêu huấn luyện cụ thể có thể rất khác nhau. Khó khăn chính là không có một mục tiêu tường minh dành riêng cho việc giám sát biểu diễn vector từ. Giải pháp thẳng nhất là tận dụng các bài toán NLP đã định nghĩa rõ. Ví dụ, dùng vector từ làm biểu diễn đầu vào cho hệ thống mô hình ngôn ngữ hay phân loại văn bản. Khi đó vector từ là tham số mô hình bình thường và được tối ưu đầu-cuối.

Cách khác là thiết kế **mục tiêu huấn luyện mới**. Như thường thấy trong học máy, không gian thiết kế cho các mục tiêu này rất rộng. Vì vậy, chiến lược phổ biến là **điều chỉnh các bài toán có sẵn**. Ví dụ, mục tiêu huấn luyện của CBOW về bản chất là bài toán đoán từ tổng quát hóa, có dạng toán học giống mô hình ngôn ngữ truyền thống.

Chương 7 sẽ bàn một số mục tiêu mới bắt nguồn từ mô hình ngôn ngữ. Nói chặt chẽ, các bài toán huấn luyện này vốn không nhằm học vector từ; chúng cung cấp một **mục tiêu thay thế (surrogate objective)** được định nghĩa rõ cho việc học biểu diễn. Hơn nữa, thường

không thể giả định học có giám sát thuần túy: embedding học được sẽ được triển khai ở nhiều hệ thống phía sau mà lúc huấn luyện ta chưa biết. Điều này biến quá trình thành một dạng **học tự giám sát (self-supervised learning)**, không có giám sát trực tiếp từ ứng dụng đích cuối cùng. Tuy nhiên, khi có ứng dụng đích và dữ liệu có nhãn cho bài toán đó, ta có thể **tinh chỉnh (fine-tune)** thêm các embedding đã tiền huấn luyện để nắm bắt ngữ nghĩa riêng của lĩnh vực.

Chú giải – Học tự giám sát: máy tự tạo “đề bài có đáp án” từ dữ liệu thô, không cần con người gán nhãn. Ví dụ: che một từ trong câu rồi bắt máy đoán – đáp án chính là từ bị che. Vì văn bản trên Internet là vô tận, cách này cho phép huấn luyện trên lượng dữ liệu khổng lồ. **Tinh chỉnh** là bước học thêm một chút trên dữ liệu chuyên ngành (ví dụ văn bản y khoa), giống như một sinh viên giỏi kiến thức chung đi thực tập để làm quen nghề cụ thể.

3.6 Đánh giá word embedding

Có được biểu diễn vector của từ rồi, ta phải đánh giá chất lượng của chúng. Lý tưởng là so với một “đáp án chuẩn”. Tiếc là đáp án chuẩn như vậy hầu như không tồn tại, vì không thể bắt người gán trực tiếp các giá trị số liên tục nhiều chiều để mô tả nghĩa của từ. Giải pháp thực tế là đánh giá vector dựa trên hiệu năng của một hệ thống thực dùng chúng. Nhìn chung, có hai loại cách đánh giá [Schnabel và cộng sự, 2015b]:

- **Đánh giá ngoại tại (Extrinsic Evaluation, hay kiểm thử đầu-cuối):** đưa vector từ vào một hệ thống NLP phía sau và đo ảnh hưởng tới hiệu năng tổng thể.
- **Đánh giá nội tại (Intrinsic Evaluation):** kiểm tra khả năng của vector từ trong việc nắm bắt các tính chất hình thái, cú pháp và ngữ nghĩa cụ thể của ngôn ngữ.

Chú giải: Giống đánh giá một học sinh: **ngoại tại** = xem học sinh đó làm việc thực tế có tốt không; **nội tại** = cho làm các bài kiểm tra chuyên biệt (từ đồng nghĩa, phép so sánh...).

3.6.1 Đánh giá ngoại tại

Cách này được dùng rộng rãi vì cho phép nhà nghiên cứu và kỹ sư nhanh chóng xem một hệ thống thực thay đổi ra sao khi biểu diễn bên dưới thay đổi. Vì nhiều hệ thống NLP nhận từ làm đầu vào, cách chuẩn là thay biểu diễn ký hiệu của từ bằng vector từ dày đặc. Ta đã

gặp các hệ thống như vậy: chúng thường dùng một lớp embedding để biến biểu diễn one-hot của mỗi từ đầu vào thành vector số thực (ví dụ mô hình ngôn ngữ nơ-ron ở Chương 2).

Cho một hệ thống như vậy và một bộ vector từ tiền huấn luyện, ta có thể dùng hiệu năng của hệ thống làm **thước đo gián tiếp** cho chất lượng embedding. Tùy cách dùng vector từ, có hai kiểu huấn luyện chính:

- **Vector từ là tham số cố định:** “đóng băng” vector từ và chỉ huấn luyện các tham số còn lại của hệ thống như thường.
- **Vector từ là tham số khởi tạo (tĩnh chỉnh):** khởi tạo lớp embedding bằng vector tiền huấn luyện và cập nhật chúng cùng mọi tham số khác của mô hình trong lúc huấn luyện.

Cả hai đều tận dụng ý tưởng tiền huấn luyện, và mở rộng được cho nhiều tình huống mà một phần mô hình được huấn luyện kỹ trước khi áp dụng cho bài toán phía sau. Đóng băng vector từ làm việc huấn luyện đơn giản hơn và tách riêng được việc đánh giá bản thân biểu diễn. Ngược lại, coi vector từ là tham số học được làm việc huấn luyện phức tạp hơn nhưng cho phép mô hình tĩnh chỉnh biểu diễn, thích nghi riêng với hệ thống đích.

Lưu ý dù đánh giá ngoại tại rất thực tế, kết quả của nó **phụ thuộc mạnh vào kiến trúc và bài toán** cụ thể. Vì phát triển hệ thống NLP hàng đầu đòi hỏi kiến trúc phức tạp và tĩnh chỉnh siêu tham số kỹ lưỡng, sự thay đổi hiệu năng không phải lúc nào cũng quy được cho chất lượng embedding. Sự phụ thuộc vào bài toán này phổ biến trong NLP. Chẳng hạn, một phương pháp tách từ rất tốt cho hệ thống dịch máy có thể kém trong hệ thống truy xuất thông tin. Vì vậy, để kiểm tra nghiêm túc khả năng tổng quát của một bộ vector từ, cách chuẩn là đánh giá chúng trên **nhiều bài toán phía sau khác nhau**.

3.6.2 Đánh giá nội tại

Dù phần lớn nghiên cứu về biểu diễn từ dùng kiểm thử đầu-cuối trong ứng dụng NLP, cũng cần kiểm tra khả năng của biểu diễn với một số vấn đề cụ thể, như diễn giải quan hệ giữa hai từ. Có nhiều cách thiết kế đánh giá nội tại, mỗi cách nhằm vào một vấn đề. Dưới đây là một số phương pháp; tổng quan đầy đủ hơn xem [Baroni và cộng sự, 2014; Bakarov, 2018; Rogers và cộng sự, 2018].

1. Độ liên quan ngữ nghĩa (Semantic Relatedness)

Mô hình hóa độ liên quan giữa các từ có lẽ là cách phổ biến nhất để đánh giá chất lượng vector từ trong NLP [Reisinger và Mooney, 2010; Huang và cộng sự, 2012; Baroni và cộng sự, 2014]. Về cơ bản, việc này gồm tính **khoảng cách không gian** giữa các từ (gọi là khoảng cách ngữ nghĩa từ, hay khoảng cách từ). Động cơ là khoảng cách hình học trong không

gian vector nên tương quan với đánh giá của con người về độ liên quan ngữ nghĩa [Rubenstein và Goodenough, 1965]. Ví dụ, ta kỳ vọng vector *dog* (chó) gần *wolf* (sói), còn *peach* (đào) thì xa *television* (ti vi).

Về toán học, có nhiều độ đo khoảng cách hay góc giữa hai vector. Đơn giản và phổ biến là **khoảng cách Euclid**. Ngoài ra, **độ tương đồng cosin** giữa hai vector cho giá trị trong đoạn $[-1, 1]$. (Trong thực tế, người ta hay đổi sang **khoảng cách cosin** $= 1 - \cos(\theta)$: bằng 0 nghĩa là hai vector cùng hướng hoàn toàn, bằng 1 nghĩa là vuông góc.)

Trong cách đánh giá này, ta có một bộ dữ liệu các cặp từ, mỗi cặp được người gán một điểm độ liên quan. Với mỗi cặp, ta so điểm của người với khoảng cách hình học đo trong không gian vector. Chất lượng vector từ được phản ánh qua **độ mạnh của tương quan** này. Tuy nhiên, thách thức chính là không có thước đo chuẩn tuyệt đối cho khoảng cách ngữ nghĩa; ngay cả người gán cũng khó định lượng chính xác mức gần nghĩa bằng con số. Một cách thực tế là quy khoảng cách về các mức rời rạc hay thang điểm kiểu **Likert**, ví dụ số nguyên từ 1 đến 5 [Reisinger và Mooney, 2010], giúp giảm đáng kể gánh nặng cho người gán nhãn. Một biến thể khác yêu cầu mô hình chọn từ giống nhất với một từ đích trong một nhóm nhỏ ứng viên (ví dụ bài kiểm tra từ đồng nghĩa). Cách này tránh phải dự đoán giá trị tuyệt đối; chỉ cần xếp hạng độ tương đồng tương đối [Baroni và cộng sự, 2014].

Chú giải: Ví dụ một bộ dữ liệu: (chó, sói) – người chấm 4,5/5; (đào, ti vi) – 0,3/5. Nếu mô hình cho $\cos(\text{chó, sói}) = 0,8$ và $\cos(\text{đào, ti vi}) = 0,05$ thì thứ tự khớp với người – tốt. Ta làm vậy với hàng nghìn cặp rồi tính hệ số tương quan (thường là Spearman) giữa điểm của người và điểm của mô hình. **Thang Likert** là kiểu thang điểm 1–5 quen thuộc trong các phiếu khảo sát (“hoàn toàn không đồng ý” $\rightarrow$ “hoàn toàn đồng ý”).

Tuy vậy, đánh giá quan hệ giữa các từ có thể mơ hồ vì ngôn ngữ vốn linh hoạt. Nhìn chung, nhiều yếu tố ngôn ngữ ảnh hưởng tới đánh giá của con người về độ liên quan [Faruqui và cộng sự, 2016]. Ví dụ, *corn* (ngô) và *cornea* (giác mạc) giống nhau về hình thái do trùng một phần chữ, nhưng nghĩa hoàn toàn khác. Sự mơ hồ cũng đến từ chính định nghĩa bài toán. **Độ liên quan ngữ nghĩa (semantic relatedness)** và **độ tương đồng ngữ nghĩa (semantic similarity)** thường được dùng thay nhau, nhưng là hai khái niệm khác nhau. Ví dụ, *car* (ô tô) **liên quan** về chủ đề với *road* (con đường), nhưng theo nghĩa phân loại chặt chẽ, *car* **tương đồng** với *van* (xe tải nhỏ).

Hơn nữa, nghĩa của từ nói chung phụ thuộc ngữ cảnh, gây nhiều cho việc đánh giá các từ có nhiều nghĩa riêng biệt (**đa nghĩa – polysemy**). Rộng hơn, đây là giới hạn cố hữu của các mô hình vector từ **tĩnh**, nơi mỗi từ luôn ứng với đúng một vector cố định. Để giải quyết điều

này, các chương sau sẽ giới thiệu **embedding theo ngữ cảnh (contextualized word embeddings)**, sinh ra dạng tính toán riêng cho từ dựa hoàn toàn vào văn bản xung quanh.

2. Phép loại suy từ (Word Analogy)

Bài toán loại suy từ quan tâm tới việc mô hình hóa **quan hệ tương tự** giữa các cặp từ. Nó xuất phát từ tiền đề rằng quan hệ ngữ nghĩa giữa các từ có thể được nắm bắt bằng các phép toán đại số đơn giản trên biểu diễn liên tục của chúng. Ví dụ kinh điển do Mikolov và cộng sự [2013d] đưa ra cho thấy ngữ nghĩa quan hệ có thể mô tả bằng phép trừ vector. Cụ thể, lấy vector *king* (vua) trừ vector *man* (đàn ông) rồi cộng vector *woman* (phụ nữ) thì được một vector rất gần vector *queen* (nữ hoàng):

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}} \quad (3.61)$$

Một cách hình thức, loại suy từ là bài toán so sánh hai cặp từ (a, a^*) và (b, b^*) . Phép loại suy đúng nếu quan hệ ngữ nghĩa giữa a và a^* giống quan hệ giữa b và b^* . Điều này phản ánh các **quy luật ngôn ngữ** tiềm ẩn trong không gian vector, viết được thành:

$$\mathbf{e}_{a^*} - \mathbf{e}_a \approx \mathbf{e}_{b^*} - \mathbf{e}_b \quad (3.62)$$

Như vậy, loại suy từ có thể coi là bài toán **suy luận loại suy**: dự đoán e_{b^*} khi biết e_a, e_{a^*} và e_b . Cụ thể, ta kỳ vọng vector $e_{a^*} - e_a + e_b$ xấp xỉ e_{b^*} nếu (a, a^*) và (b, b^*) có quan hệ ngôn ngữ giống nhau. Có nhiều cách cải tiến công thức này. Ví dụ, thay vì chỉ tối ưu khoảng cách để dự đoán e_{b^*} , có thể xét góc cosin giữa hai vector hiệu $e_{a^*} - e_a$ và $e_{b^*} - e_b$ để nắm bắt độ tương đồng quan hệ chắc chắn hơn [Levy và Goldberg, 2014b].

Chú giải – Phép màu “vua – đàn ông + phụ nữ”: Vector hiệu “vua – đàn ông” có thể hiểu là hướng “hoàng gia”. Cộng hướng đó vào “phụ nữ” ra “nữ hoàng”. Tương tự, vector “Paris – Pháp” $\approx$ “Hà Nội – Việt Nam” (hướng “thủ đô của”), hay “đi – đã đi” $\approx$ “ăn – đã ăn”. Hình dung trong mặt phẳng: mũi tên từ “đàn ông” tới “vua” song song và gần bằng mũi tên từ “phụ nữ” tới “nữ hoàng” – như hai cạnh đối của một hình bình hành. Điều đáng kinh ngạc là không ai dạy máy khái niệm “giới tính” hay “thủ đô”; nó tự xuất hiện từ việc đọc văn bản.

Bài toán loại suy từ cho một cách thẳng thắn để kiểm tra **tính chất đại số tuyến tính** của mô hình vector từ. Các đặc tính này hiếm thấy ở phương pháp dựa trên đếm truyền thống. Đáng chú ý, các mô hình word embedding hiện đại thể hiện hành vi tuyến tính mạnh dù ràng buộc hình học đó không được áp đặt khi huấn luyện. Tính chất **tự nảy sinh**

(**emergent**) này giúp nhà nghiên cứu hiểu cấu trúc ẩn mà mô hình học được và ứng dụng tiềm năng của chúng [Levy và Goldberg, 2014b; Linzen, 2016; Allen và Hospedales, 2019]. Tuy nhiên, độ chính xác loại suy từ **không phải chỉ số đại diện cho mọi trường hợp**. Trong nhiều tình huống, nó không tương quan mạnh với hiệu năng của hệ thống NLP phía sau. Vì vậy, nó thường được dùng như công cụ phân tích để thăm dò tính chất cấu trúc và ngữ nghĩa của biểu diễn từ.

3. Phân loại (hay phân cụm) từ

Một cách khác để kiểm tra vector từ có khớp với hiểu biết ngữ nghĩa của con người không là xem chúng có thể **gom thành các nhóm có nghĩa** hay không. Thường làm bằng cách áp dụng các thuật toán **phân cụm (clustering)** tiêu chuẩn lên vector từ. Lý tưởng là các từ giống nghĩa được xếp cùng cụm, còn từ khác nghĩa ở các cụm khác nhau. Ví dụ *apple*, *grape*, *peach*, *orange* (táo, nho, đào, cam) nên tự nhiên gom vào nhóm ngữ nghĩa lớn là trái cây. Ưu điểm là nhiều thuật toán phân cụm và bộ đánh giá phân loại từ đã có sẵn để dùng. Tuy nhiên, như hầu hết bài toán phân cụm không giám sát, phải giải quyết một số thách thức thực tế, như xác định trước số cụm tối ưu.

Vì hầu hết phương pháp phân cụm trong học máy dựa trên khoảng cách không gian giữa các điểm dữ liệu, phân cụm từ gắn liền với khái niệm độ liên quan ngữ nghĩa. Nhưng khác với các bài toán ở mục trên, phân cụm đánh giá **cấu trúc toàn cục** chứ không xét riêng từng điểm khoảng cách theo cặp. Điều này cho thấy mối liên hệ nội tại giữa các cách đánh giá, nhưng cũng có nghĩa phân cụm từ chịu các hạn chế cố hữu của những phương pháp liên quan. Đáng chú ý nhất là khó lập ra một tiêu chí chuẩn tuyệt đối cho chất lượng phân cụm, vì cách phân loại ngữ nghĩa mang tính chủ quan, và từ có thể được phân loại theo nhiều cách tùy góc nhìn.

Chú giải: Ví dụ, “cà chua” xếp vào nhóm “trái cây” (theo thực vật học) hay “rau” (theo ẩm thực)? “Cá heo” vào nhóm “động vật biển” hay “động vật có vú”? Cả hai cách đều đúng, nên rất khó chấm điểm tuyệt đối.

4. Đánh giá tiềm thức (Subconscious Evaluation)

Mục tiêu của đánh giá tiềm thức (hay đánh giá nhận thức) là xem xét tương quan giữa việc dùng vector từ với **hành vi tiềm thức** hay **hoạt động não bộ** của con người khi đọc. Nhiều hiện tượng tâm lý ngôn ngữ học có thể làm thước đo gián tiếp [Mitchell và Lapata, 2010]. Một mô hình nổi tiếng là **môi (priming)**, nghiên cứu cách người phản ứng với các kích thích nổi tiếp [Schacter và Buckner, 1998; Tulving và Schacter, 1990; Wiggs và Martin, 1998]. Ví dụ, thí nghiệm có thể đo tốc độ người tham gia đọc một từ (gọi là **từ đích**) ngay sau một từ

khác (gọi là **từ môi**) [Meyer và Schvaneveldt, 1971; Lund, 1995; McNamara, 2005]. Nếu từ đích t được xử lý nhanh hơn khi đi sau từ a so với khi đi sau từ b , điều đó cho thấy liên kết nhận thức giữa t và a mạnh hơn. Độ trễ tâm lý này có thể làm thước đo gián tiếp để đánh giá độ tương đồng ngữ nghĩa giữa các vector từ tương ứng. Để đo độ trễ khi đọc, phương pháp phổ biến là **bài đọc tự điều chỉnh tốc độ (self-paced reading)**: văn bản được chia thành từng từ hay cụm từ, người tham gia bấm nút để chuyển sang đoạn tiếp theo. Một cách khác là dùng thiết bị **theo dõi chuyển động mắt (eye-tracking)** để tự động ghi chính xác chuyển động và điểm dừng của mắt. Nhờ các kỹ thuật này, nhiều bộ dữ liệu và phương pháp đã được phát triển để nghiên cứu các vấn đề tâm lý ngôn ngữ học [Mitchell và Lapata, 2010; Hutchison và cộng sự, 2013; Lapesa và Evert, 2014; Klerke và cộng sự, 2015; Sogaard, 2016; Auguste và cộng sự, 2017].

Chú giải – Hiệu ứng môi: Nếu bạn vừa đọc chữ “bác sĩ”, bạn sẽ nhận ra chữ “ý tá” nhanh hơn một chút so với khi vừa đọc chữ “cái ghế”. Não đã được “môi” bởi một từ liên quan. Thời gian chênh lệch (vài phần trăm giây) cho biết hai từ liên quan chặt tới đâu trong đầu con người – và ta có thể so điều đó với khoảng cách giữa các vector.

Ngoài theo dõi phản ứng hành vi khi đọc, ta có thể trực tiếp theo dõi **hoạt động não** bằng các kỹ thuật chụp ảnh thần kinh như **chụp cộng hưởng từ chức năng (fMRI)** và **điện não đồ (EEG)** [Devereux và cộng sự, 2010; Sogaard, 2016; Bhattasali và cộng sự, 2020]. Ví dụ, hiểu ngôn ngữ tạo ra các mẫu kích hoạt thần kinh đặc trưng trong não. Ta có thể thử tương quan trực tiếp các mẫu kích hoạt sinh lý này với dạng tính toán của nghĩa từ mà biểu diễn vector cung cấp. Tuy nhiên, hạn chế là hiểu biết của ta về cơ chế nhận thức bên dưới còn chưa đầy đủ, nên khó liên hệ đầy đủ kết quả các nghiên cứu tâm lý ngôn ngữ này với hiệu năng của hệ thống NLP thực tế [Baroni và cộng sự, 2014; Bakarov, 2018].

5. Đánh giá dựa trên ngôn ngữ học

Đánh giá dựa trên ngôn ngữ học dựa trên giả định rằng vector từ học từ dữ liệu nên phản ánh tri thức cấu trúc và ngữ nghĩa được mã hóa trong các **tài nguyên ngôn ngữ học**. Một cách thú vị là căn chỉnh vector từ với biểu diễn của các mục từ trong từ điển [Tsvetkov và cộng sự, 2015; Acs và Kornai, 2016]. Chất lượng embedding được đo bằng tương quan giữa các vector liên tục này và các biểu diễn ngôn ngữ học rời rạc. (Ở đây, biểu diễn ngôn ngữ học thường là một vector đặc trưng làm thủ công lấy từ một tài nguyên ngôn ngữ chính thức, như một bộ từ vựng.) Ngoài từ điển chuẩn, vector từ có thể được đánh giá so với cấu trúc **mạng ngữ nghĩa** như **WordNet**. Khi đó, có thể làm giàu độ đo đánh giá bằng các thuật toán đồ thị để đo mức khớp cấu trúc giữa không gian vector và các tài nguyên ngữ nghĩa này [Agirre và cộng sự, 2009].

Chú giải – WordNet: là một “từ điển dạng mạng lưới” của tiếng Anh do các nhà ngôn ngữ học xây dựng, nối các từ bằng quan hệ như đồng nghĩa, “là một loại của” (chó → động vật có vú → động vật). Nếu các vector học được sắp xếp gần giống cấu trúc của mạng này thì chúng có chất lượng tốt.

3.6.3 Trục quan hóa

Coi vector từ là các điểm dữ liệu nhiều chiều, ta có thể dùng kỹ thuật giảm chiều để chiếu chúng xuống không gian 2 hoặc 3 chiều để **vẽ ra và quan sát**. Các hình chiếu này giúp phân tích trực quan các mẫu ngữ nghĩa trong vector và khám phá quan hệ ẩn giữa các từ. Một phương pháp là **PCA**, tìm ánh xạ tuyến tính từ không gian nhiều chiều xuống ít chiều (xem Mục 3.3.3). Một phương pháp nổi tiếng khác là **t-SNE** (t-distributed stochastic neighbor embedding) [Hinton và Roweis, 2002; Van der Maaten và Hinton, 2008]. Đây là phương pháp giảm chiều **phi tuyến**, được dùng rộng rãi để trục quan hóa dữ liệu nhiều chiều. Ngoài ra, có thể dùng các phương pháp ở Mục 3.3.3 hoặc các phương pháp được thiết kế riêng cho trục quan hóa vector từ [Zhang và cộng sự, 2019b; Liu và cộng sự, 2017].

Chú giải: Nếu bạn từng thấy những hình ảnh “bản đồ từ” với các chấm màu, trong đó tên các nước tụ một chỗ, tên các con vật tụ chỗ khác – đó thường là kết quả của t-SNE. Khác với PCA (chỉ xoay và chiếu thẳng), t-SNE ưu tiên giữ đúng “ai là hàng xóm của ai”, nên các cụm hiện ra rõ hơn. Tuy nhiên, khoảng cách giữa các cụm xa nhau trên hình t-SNE không phải lúc nào cũng có ý nghĩa.

3.7 Tóm tắt

Chương này bàn hai vấn đề thú vị trong NLP: **tách từ** và **biểu diễn từ (token)**. Thứ nhất, ta giới thiệu các mô hình chia câu thành những đơn vị có nghĩa và/hoặc phù hợp với các bài toán phía sau. Thứ hai, ta giới thiệu ý tưởng mô hình vector từ, đặc biệt chú ý tới việc học cả mô hình nhiều chiều dựa trên đếm lẫn mô hình ít chiều số thực. Dù phần lớn các mô hình này đơn giản, chúng thường được dùng trong các hệ thống NLP phức tạp và là nền tảng của nhiều mô hình nâng cao, như các chương sau sẽ cho thấy.

Tách từ (hay phân đoạn) là một bước quan trọng trong NLP, thường là bước tiên xử lý cho nhiều ứng dụng [Webster và Kit, 1992]. Tuy nhiên, thuật ngữ *tokenization* hơi dễ gây hiểu lầm, vì ban đầu nó chỉ quá trình cơ bản trong khoa học máy tính là chia một chuỗi thành các chuỗi con. Trong NLP hiện đại, tách từ dựa trên khái niệm từ nhiều lĩnh vực. Về ngôn ngữ học, nó gắn chặt với hai câu hỏi cốt lõi: từ được cấu tạo về hình thái như thế nào, và

chúng tạo thành câu về cú pháp ra sao. Vì vậy, dùng lý thuyết hình thái và cú pháp để định nghĩa đơn vị cơ bản của ngôn ngữ là điều tự nhiên, dẫn tới nhiều hệ thống tách từ dựa trên quy tắc cho nhiều ngôn ngữ. Từ góc độ học máy, tách từ từ lâu được coi là bài toán học ranh giới token từ dữ liệu, theo cách có giám sát hoặc không giám sát [Mielke và cộng sự, 2021]. Cách phổ biến là gán nhãn một số văn bản đã tách từ bằng hiểu biết của con người về đơn vị ngôn ngữ cơ bản, rồi học tách từ trên dữ liệu đó (Mục 3.1.3). Gần đây, học bộ tách từ **không ràng buộc ngôn ngữ học** tỏ ra rất hứa hẹn (Mục 3.1.4). Vì ngôn ngữ tự nhiên vốn là tập hợp các dãy ký tự hay byte, cũng có thể chia câu thành ký tự hay byte [Ling và cộng sự, 2015; Lee và cộng sự, 2017]. Các phương pháp **không cần tách từ** nói chung có ích khi muốn một bộ tách từ độc lập ngôn ngữ và một quy trình xử lý văn bản đơn giản hơn.

Về mặt toán học, tách từ có thể coi là một ánh xạ từ dữ liệu đầu vào sang một dãy biến. Do đó, khái niệm tách từ có thể tổng quát hóa bằng cách bỏ giả định biến đầu vào và đầu ra phải là giá trị rời rạc. Chẳng hạn, các hệ thống xử lý ảnh và tiếng nói gần đây biến dữ liệu cảm biến liên tục (như điểm ảnh và tín hiệu âm thanh) thành dãy các “token” dạng vector [Schneider và cộng sự, 2019; Dosovitskiy và cộng sự, 2021]. Một số mở rộng thú vị còn biến dữ liệu ảnh và tiếng nói thành dãy chỉ số rời rạc, nhờ đó đưa thị giác máy tính và xử lý tiếng nói xích lại gần mô hình NLP truyền thống [Oord và cộng sự, 2017; Baevski và cộng sự, 2020; Hsu và cộng sự, 2021].

Chú giải: Ví dụ, mô hình ViT (Dosovitskiy, 2021) cắt một bức ảnh thành các ô vuông 16×16 điểm ảnh và coi mỗi ô là một “token”, giống như một từ trong câu. Các mô hình tiếng nói như wav2vec, HuBERT cắt sóng âm thành từng đoạn ngắn rồi biến mỗi đoạn thành “token”. Nhờ vậy, cùng một loại mô hình có thể xử lý cả chữ, ảnh và âm thanh.

Sau khi chia văn bản đầu vào thành các đoạn nhỏ, bước tự nhiên tiếp theo là biểu diễn chúng theo cách nắm bắt được đặc trưng bên dưới. Dù biểu diễn đơn vị ngôn ngữ bằng vector số dày đặc đã thành chuẩn thực tế trong NLP hiện đại, nghiên cứu về biểu diễn vector đã có từ những ngày đầu của ngôn ngữ học tính toán. Theo nhiều giáo trình và bài báo [Manning và Schütze, 1999; Jurafsky và Martin, 2008], ý tưởng dùng phân phối thống kê để biểu diễn nghĩa của từ – gọi là **ngữ nghĩa học phân bố (distributional semantics)** – xuất hiện từ thập niên 1950 cùng với sự trỗi dậy của chủ nghĩa kinh nghiệm. Thời kỳ đó, nhiều nghiên cứu chịu ảnh hưởng mạnh của thuyết phân bố của Harris [Harris, 1954] và các khung triết học liên quan [Firth, 1957; Wittgenstein, 1953]. Cùng lúc, Osgood [1952] đề xuất định nghĩa nghĩa của một khái niệm về mặt tâm lý học, như **một điểm trong không gian nhiều chiều**. Các ý tưởng này định hình mạnh mẽ cách các nhà ngôn ngữ học và nghiên cứu NLP hình dung về nghĩa của từ trong nhiều thập kỷ sau.

Các cách tiếp cận hiện đại về ngữ nghĩa học phân bố xuất hiện vào thập niên 1990, chủ yếu nhờ sự hồi sinh của chủ nghĩa kinh nghiệm trong trí tuệ nhân tạo [Church, 2011]. Phần lớn dựa trên giả thuyết phân bố: các từ có nghĩa giống nhau có khả năng xuất hiện trong ngữ cảnh giống nhau cao hơn. Nhiều phương pháp đã được phát triển, khác nhau ở cách mô hình hóa ngữ cảnh. Ví dụ, ngữ cảnh có thể là các từ trong một cửa sổ, hoặc các từ có quan hệ với từ đang xét trong cây cú pháp. Ngoài các phương pháp ở Mục 3.3, những phương pháp không được đề cập trong chương này gồm: không gian tương tự ngôn ngữ (hyperspace analogue of language – HAL) [Lund và Burgess, 1996], bộ nhớ phân bố [Baroni và Lenci, 2010], mô hình không gian ngữ nghĩa dựa trên quan hệ phụ thuộc [Padó và Lapata, 2007]... Tổng quan đầy đủ về ngữ nghĩa học phân bố xem [Lenci, 2018; Mitchell và Lapata, 2010]. Lưu ý phần lớn các nghiên cứu trên có thể coi là trường hợp của **mô hình không gian vector**, có thể giải quyết các vấn đề vượt ra ngoài ngữ nghĩa từ vựng. Chẳng hạn, trong **ngữ nghĩa học phân bố kết hợp (compositional distributional semantics)**, nghĩa của một cụm từ hay câu được suy ra bằng các phép toán đại số có hệ thống trên các vector từ thành phần [Clark và cộng sự, 2008; Mitchell và Lapata, 2010; Blacoe và Lapata, 2012].

Dù các mô hình phân bố đã thu hút cộng đồng NLP trong nhiều năm, các mô hình word embedding học vector từ ít chiều, số thực trực tiếp từ văn bản đã trở thành hướng chủ đạo gần đây. Như trình bày ở Mục 3.4–3.5, thay vì xây tường minh ma trận đồng xuất hiện thưa, các mô hình này **ngâm** tận dụng giả thuyết phân bố bằng cách tối ưu một vector các thuộc tính ẩn để dự đoán ngữ cảnh xung quanh. Mô hình thu được là một mở rộng của **mô hình ngữ nghĩa dựa trên đặc trưng** [Markman, 2013]. Khác biệt được công nhận so với phương pháp dựa trên đặc trưng truyền thống là ta **không cần tự định nghĩa đặc trưng**, mà coi chúng là tham số của mô hình và huấn luyện theo kiểu đầu-cuối.

Đặt biểu diễn từ thành một bài toán học đầu-cuối mang lại nhiều lợi ích. Ưu điểm chính là có thể tìm ra **đặc trưng mới**, vì không có ràng buộc nào về cách các đặc trưng được học và diễn giải. Hơn nữa, như Mục 3.6.2 cho thấy, vector từ thu được theo cách này thực sự thể hiện một số tính chất ngôn ngữ học, dù mô hình word embedding không được huấn luyện để đạt điều đó. Một lợi ích khác là mô hình word embedding cũng thuộc khuôn khổ rộng của mô hình không gian vector, nên vector từ dễ dàng được dùng trong nhiều ứng dụng. Cũng có nhiều phương pháp cải tiến hệ thống word embedding tiêu chuẩn. Các nhà nghiên cứu đã thành công trong việc đưa tri thức ngôn ngữ học bên ngoài vào embedding tiền huấn luyện [Levy và Goldberg, 2014a; Cotterell và Schütze, 2015; Tissier và cộng sự, 2017] và căn chỉnh các không gian vector để tạo ra biểu diễn chung cho **nhiều ngôn ngữ** [Klementiev và cộng sự, 2012; Mikolov và cộng sự, 2013b; Ammar và cộng sự, 2020; Smith và cộng sự, 2017; Artetxe và cộng sự, 2017].

Gắn liền với các mô hình nơ-ron trong NLP, ý tưởng biểu diễn phân tán đã được tổng quát thành công cho các đơn vị cấu trúc vượt xa từ đơn lẻ. Điều này bao gồm học biểu diễn dày đặc cho **câu** [Le và Mikolov, 2014; Kalchbrenner và cộng sự, 2014; Kiros và cộng sự, 2015; Hill và cộng sự, 2016; Arora và cộng sự, 2017; Lin và cộng sự, 2017; Conneau và cộng sự, 2017] cũng như cho các cấu trúc **cây và đồ thị** phức tạp [Socher và cộng sự, 2011; Perozzi và cộng sự, 2014; Tai và cộng sự, 2015; Grover và Leskovec, 2016]. Đặc biệt, **biểu diễn từ theo ngữ cảnh** – dù không được bàn trong chương này – nói chung được đánh giá cao trong việc mô hình hóa dữ liệu tuần tự [McCann và cộng sự, 2017; Peters và cộng sự, 2018; Devlin và cộng sự, 2019].

Chú giải – Toàn cảnh chương trong 5 ý:

1. Máy tính chỉ hiểu số, nên phải **cắt văn bản thành token** (từ hoặc từ con) rồi **biến mỗi token thành vector**.
2. Cắt văn bản có thể dùng **quy tắc, mô hình xác suất**, hoặc **gán nhãn từng ký tự**; hiện đại nhất là **từ con** (BPE, WordPiece, SentencePiece) – thứ mà ChatGPT, Claude... đang dùng.
3. Vector **one-hot** chỉ cho danh tính, không cho nghĩa. Vector tốt phải làm cho **từ gần nghĩa thì gần nhau**.
4. Cách cũ: **đếm** xem từ hay đi với từ nào (ma trận đồng xuất hiện, PMI, TF-IDF) rồi **giảm chiều** (SVD/LSA, PCA). Cách mới: để **mạng nơ-ron tự học** vector qua bài tập đoán từ (Word2Vec, GloVe).
5. Đánh giá vector bằng cách xem nó giúp **bài toán thực tế** tốt lên không (ngoại tại), hoặc bằng các **bài kiểm tra** như độ tương đồng, phép loại suy “vua – đàn ông + phụ nữ $\approx$ nữ hoàng” (nội tại).

Bảng thuật ngữ Anh – Việt

Tiếng Anh	Tiếng Việt	Giải thích ngắn
token / tokenization	token / tách từ	đơn vị cơ bản sau khi cắt văn bản / quá trình cắt
word segmentation	phân đoạn từ	tách từ cho ngôn ngữ không có dấu cách giữa các từ
lemma / lemmatization	nguyên mẫu / chuẩn hóa về nguyên mẫu	dạng từ điển của từ (ran $\rightarrow$ run)

Tiếng Anh	Tiếng Việt	Giải thích ngắn
stem / stemming	gốc từ / cắt gốc từ	cắt đuôi từ một cách máy móc (removal → remov)
subword	từ con	mảnh nhỏ hơn từ (un + happy)
BPE (Byte Pair Encoding)	mã hóa cặp byte	gộp dần cặp ký hiệu hay gộp nhất
vocabulary	từ vựng	tập mọi token mô hình biết
out-of-vocabulary (OOV)	ngoài từ vựng	từ mô hình chưa từng thấy
language model	mô hình ngôn ngữ	mô hình cho biết xác suất của một câu
n-gram / unigram	n-gram / unigram	dãy n từ liên tiếp / từng từ đơn lẻ
sequence labeling	gán nhãn chuỗi	gán nhãn cho từng phần tử của dãy
HMM / CRF	mô hình Markov ẩn / trường ngẫu nhiên có điều kiện	các mô hình gán nhãn chuỗi kinh điển
one-hot representation	biểu diễn one-hot	vector toàn 0, chỉ một vị trí bằng 1
distributed representation	biểu diễn phân tán	vector số thực dày đặc mô tả thuộc tính ẩn
distributional hypothesis	giả thuyết phân bố	từ cùng ngữ cảnh thì cùng nghĩa
word embedding	nhúng từ / vector từ	vector số thực ít chiều biểu diễn từ
co-occurrence matrix	ma trận đồng xuất hiện	bảng đếm số lần các từ đi cùng nhau
context window	cửa sổ ngữ cảnh	số từ lân cận được xét
PMI	thông tin tương hỗ theo điểm	đo hai từ đi cùng nhau nhiều hơn ngẫu nhiên bao nhiêu
TF-IDF	tần suất từ – nghịch đảo tần suất tài liệu	trọng số: quan trọng trong bài × hiếm trong kho

Tiếng Anh	Tiếng Việt	Giải thích ngắn
SVD / LSA	phân tích giá trị kỳ dị / phân tích ngữ nghĩa tiềm ẩn	tách ma trận để giảm chiều
PCA	phân tích thành phần chính	tìm các hướng dữ liệu trải rộng nhất
dimensionality reduction	giảm chiều	biểu diễn dữ liệu bằng ít con số hơn
neural network	mạng nơ-ron	mô hình học máy gồm nhiều lớp tính toán
Softmax	hàm Softmax	biến dãy số thành dãy xác suất
pre-training / fine-tuning	tiền huấn luyện / tinh chỉnh	học trước trên dữ liệu lớn / học thêm cho bài toán cụ thể
CBOW / skip-gram	túi từ liên tục / skip-gram	đoán từ giữa từ ngữ cảnh / đoán ngữ cảnh từ từ giữa
GloVe	vector toàn cục	embedding học từ thống kê đồng xuất hiện toàn cục
hyperparameter	siêu tham số	con số do người chọn, không do máy học
loss function	hàm mất mát	đo mô hình sai bao nhiêu; càng nhỏ càng tốt
maximum likelihood	hợp lý cực đại	chọn tham số làm dữ liệu thật có xác suất cao nhất
EM algorithm	thuật toán Kỳ vọng - Cực đại hóa	đoán rồi sửa dần các tham số
self-supervised learning	học tự giám sát	máy tự tạo bài tập có đáp án từ dữ liệu thô
polysemy / homonymy	đa nghĩa / đồng âm	một từ nhiều nghĩa liên quan / khác nguồn gốc
word sense disambiguation (WSD)	khử nhập nhằng nghĩa từ	xác định nghĩa đúng của từ trong câu

Tiếng Anh	Tiếng Việt	Giải thích ngắn
compositionality / contextuality	tính kết hợp / tính ngữ cảnh	nghĩa tổng = ghép nghĩa thành phần / nghĩa phụ thuộc ngữ cảnh
extrinsic / intrinsic evaluation	đánh giá ngoại tại / nội tại	qua bài toán thực tế / qua bài kiểm tra chuyên biệt
cosine similarity	độ tương đồng cosin	cos góc giữa hai vector
word analogy	phép loại suy từ	vua – đàn ông + phụ nữ $\approx$ nữ hoàng
t-SNE	t-SNE	phương pháp vẽ dữ liệu nhiều chiều lên mặt phẳng